The two-machine flow shop with maximum weighted completion time: structure, algorithms and a computational study

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

The two-machine flow shop with maximum weighted completion time ($F2 \parallel WC_{\max}$) models a line dominated by a few high-priority orders. We give a mixed-integer linear programming (MILP) formulation and prove that the problem is strongly NP-hard. An optimal schedule can be taken to be a permutation schedule with one block per weight, Johnson's order within each, and non-increasing weight order across blocks. An exact dynamic program runs in $O(nK2^K V^{3K-3})$ time, where K is the number of distinct weights and V the total processing time; we add a 2-approximation in $O(n \log n)$ time and an FPTAS for every fixed K, and for large instances we adapt NEH and develop a genetic algorithm. In 440 small and 640 large instances, the MILP model solves all small ones to optimality; on a separate $K = 2$ set, the dynamic program certifies optimality up to 80 jobs for processing times in $[1, 40]$ and $[1, 80]$, and up to 100 for $[1, 40]$, the limit being time and memory rather than correctness. On small instances the 2-approximation averages 3.5 per cent above the optimum and NEH 2.5 per cent; on large ones, against a lower bound, they average 1.8 and 4.4 per cent. The genetic algorithm has the largest gap at small sizes, but it is the only method that overtakes the other two, at roughly 240 to 280 jobs, and at a much higher cost.

Keywords: Scheduling; Two-machine flow shop; Dynamic programming; Weighted makespan;

1 Introduction

The maximum weighted completion time is a bottleneck objective in scheduling. With a few jobs carrying large weights, the schedule is judged by the job with a large weight

*Corresponding author. E-mail address: author@email.com

and a late completion. The objective differs from the makespan, the completion time of the last job. The two objectives coincide for equal weights; with unequal weights, the weighted objective can favor a schedule with a larger makespan. Such a schedule serves the important jobs earlier. This paper studies the two-machine flow shop; Johnson's rule minimizes the makespan (Johnson, 1954). That rule ignores weights.

Our objective is the maximum weighted completion time, also called the weighted makespan. Each job carries a weight, and its weighted completion time is the product of that weight and its completion time. The objective is the largest of these values: a schedule is judged by its worst job, and one job can decide the value of the whole schedule. The objective arises in a real sequencing problem in semiconductor fabrication. A wafer lot, typically a cassette of twenty-five wafers, undergoes photolithography, etching, deposition, implantation and polishing. A chip is built layer by layer, with each layer sending the lot back through photolithography and etching. A lot may visit the photolithography area dozens of times, usually the bottleneck of the line. The lots are not equally important: a few hot lots must be expedited and take priority over regular lots, with regular lots filling the remaining capacity. A hot lot can wait behind regular lots, delaying a customer order promised for early delivery. The two-machine flow shop in this paper is a simplified version of such a line, with each lot visiting each machine once. The scheduling problem is to order the lots and minimize the largest weighted completion time. In the model, a lot's weight reflects its priority, and the worst hot lot determines the value of the schedule.

For the flow shop itself, the makespan has been studied extensively beyond Johnson's rule. Insertion heuristics and their refinements build a schedule one job at a time (Nawaz et al., 1983; Puka et al., 2024). Branch-and-bound procedures address stochastic and re-entrant shops (Liu and Urgo, 2024a,b). The same objective has also been studied in distributed, assembly and transport variants (Becker et al., 2025; Han et al., 2024; Ying et al., 2025). Weighted objectives have been studied for the sum rather than the maximum: the total weighted completion time is NP-hard already with two machines (Garey et al., 1976). The methods for it rely on additivity, absent from the maximum. Flow shops with reentry have been studied for the total weighted completion time as well (von Aspern et al., 2025). To the best of our knowledge, the maximum weighted completion time has not been studied on two machines.

Two features shape the difficulty. Two machines in series make each completion time depend on the idle time of its predecessors, and the objective is a maximum rather than a sum, so it is not additive across jobs. An exchange that shortens many completion times but lengthens the worst job does not improve the value. Johnson's rule has no direct extension to unequal weights. We will prove that the problem is strongly NP-hard, even with polynomially bounded processing times. The number of distinct weights, K, is the governing parameter: for $K = 1$, the problem reduces to the makespan; the general case is a high-multiplicity problem (Koutecký and Zink, 2025). Jobs of equal weight are therefore interchangeable.

An optimal schedule can be taken to be a permutation with one block per distinct weight, Johnson's order within each block, and the blocks in non-increasing weight order. The paper makes five contributions: (i) a mixed integer programming formulation; (ii) a proof of this hardness by a reduction from 3-Partition; (iii) this structural theorem; (iv) an exact dynamic program in $O(nK2^K V^{3K-3})$ time, a 2-approximation in $O(n \log n)$ time, and a fully polynomial-time approximation scheme for a fixed K; and (v) an adapted NEH heuristic, a genetic algorithm, and a computational study on 440 small and 640 large instances.

The rest of the paper is organized as follows. Section 2 states the problem and gives the mixed integer programming formulation. Section 3 gives the hardness proof. Section 4 presents the structural theorem, the dynamic program, the 2-approximation and the approximation scheme. Section 5 describes the adapted NEH and the genetic algorithm. Section 6 reports the computational study, and Section 7 concludes.

2 Problem description

Consider a set $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ of n independent and non-preemptive jobs to be scheduled in a two-machine flow shop with machines M_1 and M_2. All jobs are released at time zero and follow the same route, $M_1 \to M_2$. For $j = 1, 2, \ldots, n$, job J_j is represented by a triple $\Phi_j = (a_j, b_j, w_j)$, where a_j and b_j are its nonnegative integer processing times on M_1 and M_2, respectively, and w_j is a weight reflecting its importance. Let K be the number of distinct weights. For a feasible schedule π, let $C_j^1(\pi)$ and $C_j^2(\pi)$ denote the completion times of J_j on M_1 and M_2, respectively; thus $C_j(\pi) = C_j^2(\pi)$ is its completion time. The makespan and the weighted makespan (i.e., the maximum weighted completion time) in π are given by

$$C_{\max}(\pi) = \max_{1 \le j \le n} \{C_j(\pi)\} \quad \text{and} \quad WC_{\max}(\pi) = \max_{1 \le j \le n} \{w_j C_j(\pi)\},$$

respectively. The objective is to minimize $WC_{\max}$. Using the standard 3-field scheduling notation (Pinedo, 2022), this problem is denoted by $F2 \,||\, WC_{\max}$.

In a two-machine flow shop, a schedule is called a *permutation* schedule if the jobs are processed in the same order on both machines. Johnson (1954) showed that an optimal permutation schedule exists for $F2 \,||\, C_{\max}$. By a similar argument, an optimal permutation schedule also exists for $F2 \,||\, WC_{\max}$. Hence, we restrict attention to permutation schedules. Let $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ denote a feasible permutation schedule for $F2 \,||\, WC_{\max}$, where $J_{\pi(j)}$ is the j-th job in π. By simple calculation, it follows that

$$C_{\pi(j)} = \max_{1 \le i \le j} \left\{ \sum_{h=1}^{i} a_{\pi(h)} + \sum_{h=i}^{j} b_{\pi(h)} \right\}, \quad j = 1, 2, \ldots, n. \tag{1}$$

Then, the weighted makespan of π is given by

$$WC_{\max}(\pi) = \max_{1 \le j \le n} \{w_{\pi(j)} C_{\pi(j)}\}. \tag{2}$$

To formulate the problem as a mixed integer linear program (MILP), we introduce the following decision variables:

- $x_{jk} = 1$ if job J_j is the k-th job, and 0 otherwise;
- $C^1_{[k]}$ is the completion time of the k-th job on M_1;
- $C^2_{[k]}$ is the completion time of the k-th job on M_2;
- C_j is the completion time of job J_j;
- M is a sufficiently large constant (e.g., $M = \sum_{j=1}^{n}(a_j + b_j)$).

The MILP model can be formulated as follows:

$$\min \quad WC_{\max} \tag{3}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad j = 1, 2, \ldots, n \tag{4}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad k = 1, 2, \ldots, n \tag{5}$$

$$C^1_{[1]} = \sum_{j=1}^{n} a_j x_{j1} \tag{6}$$

$$C^1_{[k]} = C^1_{[k-1]} + \sum_{j=1}^{n} a_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{7}$$

$$C^2_{[1]} = C^1_{[1]} + \sum_{j=1}^{n} b_j x_{j1} \tag{8}$$

$$C^2_{[k]} \geq C^1_{[k]} + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{9}$$

$$C^2_{[k]} \geq C^2_{[k-1]} + \sum_{j=1}^{n} b_j x_{jk}, \quad k = 2, 3, \ldots, n \tag{10}$$

$$C_j \geq C^2_{[k]} - M(1 - x_{jk}), \quad j = 1, 2, \ldots, n; \quad k = 1, 2, \ldots, n \tag{11}$$

$$WC_{\max} \geq w_j C_j, \quad j = 1, 2, \ldots, n \tag{12}$$

$$x_{jk} \in \{0, 1\}, \quad j = 1, 2, \ldots, n; \quad k = 1, 2, \ldots, n \tag{13}$$

$$C_j, C^1_{[k]}, C^2_{[k]} \geq 0 \quad j = 1, 2, \ldots, n; \quad k = 1, 2, \ldots, n \tag{14}$$

The objective is to minimize $WC_{\max}$, as expressed in (3). Constraints (4) and (5) assign each job to exactly one position and each position to exactly one job. Constraints (6) and (7) determine the completion times on M_1, and (8) through (10) determine those those on M_2. In particular, for $k \geq 2$, the job in position k can start on M_2 only after it has completed M_1 and the preceding job has finished on M_2. Constraint (11) relates the position-based

completion times to the job-based ones via a big-M formulation. Note that when $x_{jk} = 1$ (i.e., J_j is assigned to the k-th position), it requires $C_j \geq C^2_{[k]}$; when $x_{jk} = 0$, it is inactive, since $C^2_{[k]} - M \leq 0$ and $C_j \geq 0$. Constraint (12) imposes $WC_{\max} \geq w_j C_j$ for each job, so minimizing $WC_{\max}$ yields the optimal objective value of (2). Constraints (13) and (14) require the decision variables x_{jk} to be binary and the completion times nonnegative.

3 Computational complexity

When all jobs have unit weights (i.e., $w_j = 1$ for $j = 1, 2, \ldots, n$), $F2 \,||\, WC_{\max}$ reduces to $F2 \,||\, C_{\max}$, which is solvable in $O(n \log n)$ time (Johnson, 1954). In contrast, We show that $F2 \,||\, WC_{\max}$ is ordinarily $\mathcal{NP}$-hard even with only two distinct weights and strongly $\mathcal{NP}$-hard when K is arbitrary.

Theorem 3.1. *$F2 \,||\, WC_{\max}$ is ordinarily $\mathcal{NP}$-hard even with only two distinct weights.*

Proof. To establish the result, we give a polynomial-time reduction from PARTITION, which is $\mathcal{NP}$-complete in the ordinary sense (Garey and Johnson, 1979).

PARTITION: Given r positive integers $x_1, x_2, \ldots, x_r$ and an integer X with $\sum_{i=1}^{r} x_i = 2X$, does there exist a partition of $\{1, 2, \ldots, r\}$ into two disjoint subsets $\mathcal{X}_1$ and $\mathcal{X}_2$ such that $\sum_{i \in \mathcal{X}_1} x_i = \sum_{i \in \mathcal{X}_2} x_i = X$?

Given an arbitrary instance $(x_1, x_2, \ldots, x_r; X)$ of PARTITION, we construct an instance of $F2 \,||\, WC_{\max}$ with $n = r + 1$ jobs. For $j = 1, 2, \ldots, n-1$, job J_j has $a_j = 1$, $b_j = rx_j$, and $w_j = X + 1$; job J_n has $a_n = rX$, $b_n = 1$, and $w_n = 2X + 1$. Set the threshold to $Y = r(X + 1)(2X + 1)$. The question is whether there exists a schedule with $WC_{\max} \leq Y$.

We demonstrate that $F2 \,||\, WC_{\max}$ has a feasible schedule π with $WC_{\max}(\pi) \leq Y$ if and only if PARTITION has a solution.

Suppose that PARTITION admits a solution $(\mathcal{X}_1, \mathcal{X}_2)$ with $\sum_{j \in \mathcal{X}_1} x_j = \sum_{j \in \mathcal{X}_2} x_j = X$. Define $\mathcal{J}_1 = \{J_j : j \in \mathcal{X}_1\}$ and $\mathcal{J}_2 = \{J_{r+j} : j \in \mathcal{X}_2\}$. We build a feasible permutation schedule π in which both machines process the jobs in the order $\mathcal{J}_1 \to J_n \to \mathcal{J}_2$, with the jobs in $\mathcal{J}_1$ and $\mathcal{J}_2$ processed in arbitrary order (see Figure 1).

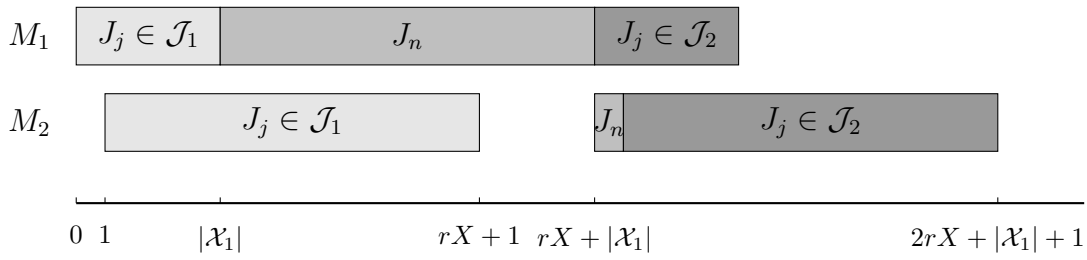


Figure 1: A feasible permutation scheule π

Since $a_j \le b_j$ for each $j = 1, 2, \ldots, r$ and $|\mathcal{X}_1| \le r - 1$, it can be easily verified that $C_n(\pi) = rX + |\mathcal{X}_1| + 1 \le r(X+1)$ and $C_{\max}(\pi) = 2rX + |\mathcal{X}_1| + 1 \le r(2X+1)$. Hence, $WC_{\max}(\pi) \le r(X+1)(2X+1) = Y$.

Conversely, suppose that $F2 \,||\, WC_{\max}$ admits a feasible schedule π with $WC_{\max}(\pi) \le Y$. Let $\mathcal{J}_1 = \{J_j : C_j(\pi) < C_n(\pi)\}$ and $\mathcal{J}_2 = \{J_j : C_j(\pi) > C_n(\pi)\}$ be the sets of jobs completed before and after J_n in π, respectively, and let $\mathcal{X}_1 = \{j : J_j \in \mathcal{J}_1\}$ and $\mathcal{X}_2 = \{j : J_j \in \mathcal{J}_2\}$. By (1), we have

$$C_n(\pi) \ge \sum_{j \in \mathcal{X}_1} a_j + a_n + 1 = |\mathcal{X}_1| + rX + 1 \tag{15}$$

and

$$C_n(\pi) \ge a_{\pi(1)} + \sum_{j \in \mathcal{X}_1} b_j + b_n = r \sum_{j \in \mathcal{X}_1} x_j + 2. \tag{16}$$

Case 1. If $\sum_{j \in \mathcal{X}_1} x_j > X$, then $\sum_{j \in \mathcal{X}_1} x_j \ge X + 1$ because the x_j are integers. By (16), $WC_{\max}(\pi) \ge w_n C_n(\pi) \ge (2X+1)(r(X+1)+2) > r(X+1)(2X+1) = Y$, contradicting the assumption that $WC_{\max}(\pi) \le Y$.

Case 2. If $\sum_{j \in \mathcal{X}_1} x_j < X$, then $\sum_{j \in \mathcal{X}_1} x_j \le X - 1$ because the x_j are integers. Since $(\mathcal{X}_1, \mathcal{X}_2)$ is a partition of $\{1, 2, \ldots, r\}$, we have

$$\sum_{j \in \mathcal{X}_2} x_j = 2X - \sum_{j \in \mathcal{X}_1} x_j \ge X + 1. \tag{17}$$

By (1), (15), and (17), we have

$$C_{\pi(n)}(\pi) = C_{\max}(\pi) \ge C_n(\pi) + \sum_{j \in \mathcal{X}_2} b_j \ge |\mathcal{X}_1| + rX + 1 + r \sum_{j \in \mathcal{X}_2} x_j \ge r(2X+1) + 1. \tag{18}$$

Using $w_{\pi(n)} = 1$, (18) yields

$$WC_{\max}(\pi) \ge w_{\pi(n)} C_{\pi(n)}(\pi) \ge (X+1)(r(2X+1)+1) > r(X+1)(2X+1) = Y,$$

This contradicts the assumption that $WC_{\max}(\pi) \le Y$.

Combining the above two cases, we obtain that $\sum_{j \in \mathcal{X}_1} x_j = \sum_{j \in \mathcal{X}_2} x_j = X$. Therefore, $(\mathcal{X}_1, \mathcal{X}_2)$ is a solution to PARTITION. □

Theorem 3.2. *$F2 \,||\, WC_{\max}$ is strongly $\mathcal{NP}$-hard when K is arbitrary.*

Proof. To establish the result, we give a polynomial-time reduction from 3-PARTITION, which is strongly $\mathcal{NP}$-complete (Garey and Johnson, 1979).

3-PARTITION: Given $3r$ positive integers $x_1, x_2, \ldots, x_{3r}$ and an integer X with $\sum_{i=1}^{3r} x_i = rX$ and $\frac{X}{4} < x_j < \frac{X}{2}$ for each $j = 1, 2, \ldots, 3r$, does there exist a partition of $\{1, 2, \ldots, 3r\}$ into m disjoint subsets $\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r$ such that $\sum_{j \in \mathcal{X}_i} x_j = X$ for $i = 1, 2, \ldots, r$?

Given an arbitrary instance $(x_1, x_2, \ldots, x_{3r}; X)$ of 3-PARTITION, we construct an instance of $F2 \,||\, WC_{\max}$ consisting of r separating jobs and $3r$ basic jobs. For $j = 1, 2, \ldots, r$,

separating job J_j has $a_j = 4X$, $b_j = X$, and $w_j = \frac{5r+4}{5j}$; for $j = r+1, r+2, \ldots, 4r$, basic job J_j has $a_j = x_{j-r}$, $b_j = 4x_{j-r}$, and $w_j = 1$. Set the threshold to $Y = (5r+4)X$. The question is whether there exists a schedule with $WC_{\max} \leq Y$.

We demonstrate that $F2 \,||\, WC_{\max}$ has a feasible schedule π with $WC_{\max}(\pi) \leq Y$ if and only if 3-PARTITION has a solution.

Suppose that 3-PARTITION admits a solution $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$, so that $\sum_{j \in \mathcal{X}_i} x_j = X$ and $|\mathcal{X}_i| = 3$ for all i. Let $\mathcal{J}_i = \{J_{r+j} : j \in \mathcal{X}_i\}$ for $i = 1, 2, \ldots, r$. We build a feasible permutation schedule π in which both machines process the jobs in the order

$$J_1 \to \mathcal{J}_1 \to J_2 \to \mathcal{J}_2 \to \cdots \to J_r \to \mathcal{J}_r,$$

with the jobs in each $\mathcal{J}_i$ processed in arbitrary order (see Figure 2).

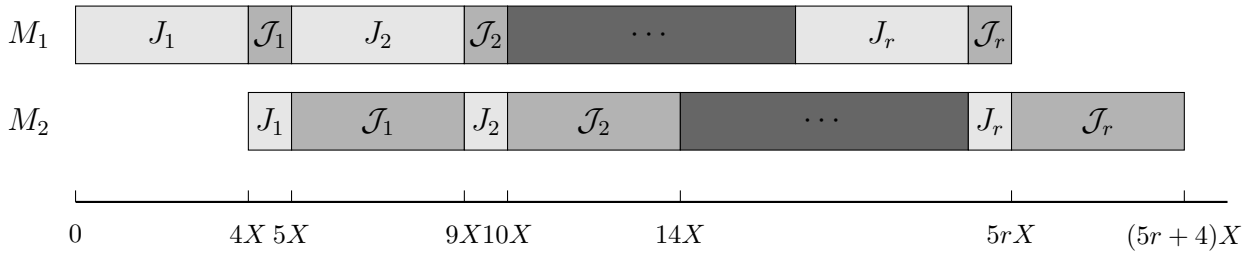


Figure 2: A feasible permutation scheule π

According to Figure 2, it can be easily verified that $C_j(\pi) = 5jX$ for $j = 1, 2, \ldots, r$, and $C_j(\pi) \leq C_{\max}(\pi) = (5r+4)X$ for $j = r+1, r+2, \ldots, 4r$. Then $w_j C_j(\pi) = (5r+4)X = Y$ for $j = 1, 2, \ldots, r$, and $w_j C_j(\pi) \leq w_j C_{\max}(\pi) = (5r+4)X = Y$ for $j = r+1, r+2, \ldots, 4r$. Hence, $WC_{\max}(\pi) \leq Y$.

Conversely, suppose that $F2 \,||\, WC_{\max}$ admits a feasible schedule π with $WC_{\max}(\pi) \leq Y$. Because all separating jobs have the same processing time on M_1 and M_2 and weights $w_1 > w_2 > \cdots > w_r$, a standard interchange argument lets us assume they appear in π in decreasing order of weight.

Claim 1. $C_{\max}(\pi) \leq Y = (5r+4)X$.

Proof of Claim 1. Since $w_j \geq 1$ for all $1 \leq j \leq 4r$, we have $C_{\max}(\pi) \leq WC_{\max}(\pi) \leq Y = (5r+4)X$. ■

Claim 2. J_1 is the first job scheduled in π.

Proof of Claim 2. Since $WC_{\max}(\pi) \leq Y = (5r+4)X$ and $w_1 = \frac{5r+4}{5}$, we have $a_1 + b_1 \leq C_1(\pi) \leq \frac{WC_{\max}(\pi)}{w_1} \leq 5X = a_1 + b_1$. This means that J_1 is the first job scheduled in π. ■

For $i = 1, 2, \ldots, r-1$, let $\mathcal{B}_i = \{J_j : C_i(\pi) < C_j(\pi) < C_{i+1}(\pi)\}$ be the set of basic jobs scheduled between J_i and J_{i+1} in π, and let $\mathcal{B}_r = \{J_j : C_j(\pi) > C_r(\pi)\}$ be the set of basic

jobs scheduled after J_r. For $i = 1, 2, \ldots, r$, define $\mathcal{X}_i = \{j - r : J_j \in \mathcal{B}_i\}$. Then, by Claim 2, π can be written as

$$\pi = (J_1 \to \mathcal{B}_1 \to J_2 \to \mathcal{B}_2 \to \cdots \to J_r \to \mathcal{B}_r). \tag{19}$$

Since $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$ is a partition of $\{1, 2, \ldots, 3r\}$, we have

$$\sum_{i=1}^{r} \sum_{j \in \mathcal{X}_i} x_j = \sum_{j=1}^{3r} x_j = rX. \tag{20}$$

Claim 3. For $k = 1, 2, \ldots, r-1$, $\sum_{i=1}^{k} \sum_{j \in \mathcal{X}_i} x_j \le kX$.

Proof of Claim 3. Suppose to the contrary that there exists some m $(1 \le m \le r-1)$ with $\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j > mX$. Since the x_j are integers, this implies

$$\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j \ge mX + 1. \tag{21}$$

Because $a_j = 4X$ and $b_j = X$ for $1 \le j \le r$, and $a_j = x_{j-r}$ for $r+1 \le j \le 4r$, (1), (19), and (21) give

$$C_{m+1}(\pi) \ge \sum_{j=1}^{m+1} a_j + \sum_{i=1}^{m} \sum_{j \in \mathcal{B}_i} a_j + b_{m+1} = 4(m+1)X + \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j + X \ge 5(m+1)X + 1. \tag{22}$$

Using $w_{m+1} = \frac{5r+4}{5(m+1)}$, (22) yields

$$WC_{\max}(\pi) \ge w_{m+1} C_{m+1}(\pi) \ge \frac{5r+4}{5(m+1)}(5(m+1)X + 1) > (5r+4)X = Y, \tag{23}$$

which contradicts the assumption that $WC_{\max}(\pi) \le Y$. Hence, Claim 3 holds. ■

Claim 4. For $k = 1, 2, \ldots, r-1$, $\sum_{i=1}^{k} \sum_{j \in \mathcal{X}_i} x_j \ge kX$.

Proof of Claim 4. Suppose to the contrary that there exists some m $(1 \le m \le r-1)$ with $\sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j < mX$. By (20) and the x_j are integers, we have

$$\sum_{i=m+1}^{r} \sum_{j \in \mathcal{X}_i} x_j = rX - \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j \ge (r-m)X + 1. \tag{24}$$

Because $a_j = 4X$ and $b_j = X$ for $1 \le j \le r$, and $a_j = x_{j-r}$ and $b_j = 4x_{j-r}$ for $r+1 \le j \le 4r$, (1), (19), (20), and (24) give

$$\begin{aligned} C_{\max}(\pi) &\ge \sum_{j=1}^{m+1} a_j + \sum_{i=1}^{m} \sum_{j \in \mathcal{B}_i} a_j + \sum_{j=m+1}^{r} b_j + \sum_{i=m+1}^{r} \sum_{j \in \mathcal{B}_i} b_j \\ &= 4(m+1)X + \sum_{i=1}^{m} \sum_{j \in \mathcal{X}_i} x_j + (r-m)X + \sum_{i=m+1}^{r} \sum_{j \in \mathcal{X}_i} 4x_j \end{aligned}$$

$$
\begin{aligned}
&= 4(m+1)X + (r-m)X + rX + 3\sum_{i=m+1}^{r}\sum_{j\in\mathcal{X}_i} x_j \\
&\geq 4(m+1)X + (r-m)X + rX + 3((r-m)X+1) \\
&= (5r+4)X + 3 > (5r+4)X = Y.
\end{aligned}
$$

which contradicts Claim 1. Hence, Claim 4 holds. ■

Combining Claims 3 and 4 with (20), we obtain that $\sum_{j\in\mathcal{X}_i} x_j = X$ for $i = 1, 2, \ldots, r$. Since $X/4 < x_j < X/2$ for each j, a sum equal to X must contain exactly three elements; hence $|\mathcal{X}_i| = 3$ for each i. Therefore, $(\mathcal{X}_1, \mathcal{X}_2, \ldots, \mathcal{X}_r)$ is a solution to 3-PARTITION. □

4 Exact and approximation algorithms

This section develops the exact dynamic program and the two approximation algorithms that rest on it. Throughout, K denotes the number of distinct weights and $V = \sum_{j=1}^{n}(a_j+b_j)$ the total processing time.

Since the objective is a maximum over weighted completion times, jobs of equal weight are interchangeable. An optimal schedule can therefore be assumed to have a block structure: one block per distinct weight, Johnson's order within each block, and blocks in non-increasing weight order. A schedule is fixed once each weight class has been assigned to a block, and the running time of every algorithm below is governed by the number K of those classes.

Three algorithms are built on this structure. The first is exact and prunes dominated states: a state records each block's processing time on both machines, the completion time of its last job on M_2, and that job's weight, and the dynamic program runs in $O(n \cdot K \cdot |\mathscr{H}|_{\max})$ time with $|\mathscr{H}|_{\max} \leq 2^K V^{3K-3}$. The second sorts the jobs by non-increasing weight in $O(n \log n)$ time and returns a schedule within a factor 2 of the optimum. The third rounds the time coordinates and the weights to geometrically spaced grids, which turns the dynamic program into a $(1+\epsilon)$-approximation scheme for a fixed number of distinct weights.

4.1 Block structure of an optimal schedule

Within the permutation schedules of Section 2, Johnson's rule (Pinedo, 2022) minimizes the makespan: the jobs with $a_j \leq b_j$ are scheduled first in non-decreasing order of a_j, and the remaining jobs follow in non-increasing order of b_j.

Fix a permutation schedule π. Let $W_1 > W_2 > \cdots > W_K$ be the K distinct weights occurring among the jobs, and let

$$
\mathcal{G}_k = \{J_j \in \mathcal{J} \mid w_j = W_k\}, \qquad k = 1, \ldots, K,
$$

denote the corresponding weight classes. Set $\ell_0 = 0$. For $k = 1, \ldots, K$ in sequence, let ℓ_k be the position of the last job of weight W_k among the jobs not yet assigned to a block, that

is, among $\pi(\ell_{k-1}+1),\dots,\pi(n)$; if no such job remains, set $\ell_k = \ell_{k-1}$. The blocks are the consecutive segments

$$\mathcal{B}_k = \{\pi(\ell_{k-1}+1),\dots,\pi(\ell_k)\}, \qquad k = 1,\dots,K,$$

so that in particular $\mathcal{B}_1 = \{\pi(1),\dots,\pi(\ell_1)\}$. Equivalently, once the blocks $\mathcal{B}_1,\dots,\mathcal{B}_k$ have been removed, the remaining jobs are

$$\mathcal{J} \setminus \bigcup_{i=1}^{k} \mathcal{B}_i = \{\pi(\ell_k+1),\dots,\pi(n)\}, \qquad k = 1,\dots,K.$$

Hence $\ell_1 \le \ell_2 \le \cdots \le \ell_K$, and no block extends beyond its own last job. For each block $\mathcal{B}_i$, let A_i and B_i denote the total processing time of its jobs on M_1 and M_2, respectively, C_i the completion time of its last job on M_2, and L_i the weight of its last job. If $\ell_k = \ell_{k-1}$, then $\mathcal{B}_k$ contains no job; such a block is called empty and satisfies $A_k = B_k = C_k = L_k = 0$, and it may host any job of weight at most W_k. By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight W_k, whereas an empty block has $L_k = 0$. Since $W_1 > W_2 > \cdots > W_K$, the block-end weights are non-increasing on the non-empty blocks.

We illustrate the block construction with a small example.

Example 4.1. *Consider a six-job instance with sequence $\pi = (J_1,\dots,J_6)$ and the following (a_j, b_j, w_j) values:*

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	4	3
b_j	5	7	9	4	3	2
w_j	2	2	5	2	4	2

The weight classes are $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_5\}$, and $\mathcal{G}_3 = \{J_1, J_2, J_4, J_6\}$, and the resulting blocks are $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, and $\mathcal{B}_3 = \{J_6\}$.

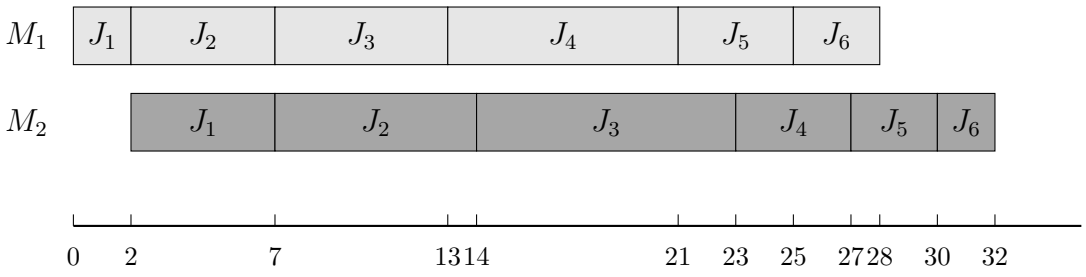


Figure 3

Lemma 4.1. *For any schedule π, scheduling the jobs of each block by Johnson's rule and concatenating the blocks in their original order gives a schedule π' with $WC_{\max}(\pi') \le WC_{\max}(\pi)$.*

Proof. By construction, a non-empty block $\mathcal{B}_k$ ends with a job of weight L_k, and every job in $\mathcal{B}_k$ has weight at most L_k: a weight occurring in $\mathcal{B}_k$ either equals W_k, or it has already appeared in an earlier block, whose weights are larger. Let $C_{\pi(\ell_k)}$ denote the completion time of the last job of $\mathcal{B}_k$ in π, the job that sits at position ℓ_k. An empty block satisfies $L_k C_{\pi(\ell_k)} = 0$ and therefore contributes nothing to the objective, so the argument below applies to the non-empty blocks.

We first show that the maximum weighted completion time within a block is attained by the last job of that block. For any $J_j \in \mathcal{B}_k$, the last job of the block completes no earlier than J_j in π, so $C_j \leq C_{\pi(\ell_k)}$, while $w_j \leq L_k$ by the block construction. Hence $w_j C_j \leq L_k C_{\pi(\ell_k)}$, and taking the maximum over $j \in \mathcal{B}_k$ gives

$$\max_{J_j \in \mathcal{B}_k} w_j C_j = L_k C_{\pi(\ell_k)}.$$

The objective of (2) therefore equals

$$WC_{\max}(\pi) = \max_{k=1,\ldots,K} L_k C_{\pi(\ell_k)},$$

so it depends only on the last job of each block.

Let π' be the schedule obtained by rescheduling the jobs within each block $\mathcal{B}_k$ according to Johnson's rule, keeping the blocks in their original order. We prove $C_{\pi'(\ell_k)} \leq C_{\pi(\ell_k)}$ for every k by induction on k.

Base case $k = 1$. Block $\mathcal{B}_1$ starts at time zero on both machines regardless of its internal order, so Johnson's rule gives $C_{\pi'(\ell_1)} \leq C_{\pi(\ell_1)}$.

Inductive step. Assume $C_{\pi'(\ell_{k-1})} \leq C_{\pi(\ell_{k-1})}$. On M_1, the start time of $\mathcal{B}_k$ equals the total M_1 work of $\mathcal{B}_1, \ldots, \mathcal{B}_{k-1}$, which is invariant under internal rescheduling, so $\mathcal{B}_k$ begins at the same instant on M_1 under both schedules. On M_2, block $\mathcal{B}_k$ starts at $\max\{\sum_{i=1}^{k} A_i,\, C_{\pi(\ell_{k-1})}\}$; the first term is identical under π and π', and the second is no larger under π' by the induction hypothesis. Hence $\mathcal{B}_k$ starts no later on M_2 under π' than under π. Together with the same M_1 start and the internally optimal Johnson scheduling, this gives $C_{\pi'(\ell_k)} \leq C_{\pi(\ell_k)}$, which completes the induction.

Finally,

$$WC_{\max}(\pi') = \max_k L_k C_{\pi'(\ell_k)} \leq \max_k L_k C_{\pi(\ell_k)} = WC_{\max}(\pi),$$

which establishes the lemma. □

4.2 An exact dynamic program

The exact dynamic program processes the jobs in the Johnson sequence and assigns each job to one of the K blocks $\mathcal{B}_1, \ldots, \mathcal{B}_K$.

Lemma 4.1 provides the structural basis for the dynamic program. Given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's

rule and concatenating the blocks in the order of non-increasing block weights yields a schedule whose $WC_{\max}$ value is no larger than that of any other schedule respecting the same assignment. For each $j = 1, \ldots, n$, let $\mathscr{H}_j$ denote the set of states attainable after the first j jobs have been processed. A state records the following quantities for every block $\mathcal{B}_i$:

- A_i: the total processing time on machine M_1 of the jobs in block $\mathcal{B}_i$;
- B_i: the total processing time on machine M_2 of the jobs in block $\mathcal{B}_i$;
- C_i: the completion time of the last job of block $\mathcal{B}_i$ on machine M_2;
- L_i: the weight of the last job of block $\mathcal{B}_i$.

Since $W_1 > \cdots > W_K$, the blocks eligible for J_j form a prefix. Writing

$$i^* = \max\{\, i : w_j \le W_i \,\},$$

the eligible set is $\mathcal{E}(J_j) = \{\, 1, \ldots, i^* \,\}$. Each state is therefore the aggregate K-block tuple

$$(A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ C_1, \ldots, C_K;\ L_1, \ldots, L_K),$$

which is described by $4K$ parameters.

The initial state set contains a single state whose components are all zero: $\mathscr{H}_0 = \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$. In Algorithm 1, T_{A_i} denotes the total M_1 processing time of the first i blocks, and T_{B_i} the completion time of block $\mathcal{B}_i$ on M_2.

Lemma 4.2. *Let $\mathscr{H} = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$ and $\mathscr{H}' = (A'_1, \ldots, A'_K; B'_1, \ldots, B'_K; C'_1, \ldots, C'_K; L'_1, \ldots, L'_K)$ be two states in $\mathscr{H}_j$. If $A_i = A'_i$, $B_i = B'_i$ and $L_i = L'_i$ for every i, and $C_i \le C'_i$ for every i, then $\mathscr{H}'$ may be discarded without changing the optimal value.*

Proof. Consider any continuation that extends $\mathscr{H}'$ to a complete schedule Π'. Replacing the prefix represented by $\mathscr{H}'$ with the prefix represented by $\mathscr{H}$ yields a schedule Π that assigns the remaining jobs identically. For every block $\mathcal{B}_k$ that follows the updated one, the M_2 start time is $\max\{\sum_{i=1}^{k} A_i,\ C_{k-1}\}$, and C_{k-1} is no larger under $\mathscr{H}$ than under $\mathscr{H}'$. By induction, the M_2 completion time of every subsequent block is no larger under $\mathscr{H}$. Since A_i, B_i and L_i are identical in the two states, the objective value $WC_{\max}$ of Π does not exceed that of Π'. □

For $j = 1, \ldots, n$, the algorithm extends $\mathscr{H}_{j-1}$ to $\mathscr{H}_j$ by placing J_j at the end of each eligible block. That is, for every state in $\mathscr{H}_{j-1}$ and every $i \in \mathcal{E}(J_j)$, only the i-th slice of the state is updated while all other blocks remain unchanged:

$$A'_i = A_i + a_j, \quad B'_i = B_i + b_j, \quad C'_i = \max\{C_i + b_j,\ A_i + a_j + b_j\}. \tag{25}$$

The resulting candidate states are collected in temporary sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$. By Lemma 4.2, merging the candidates removes dominated and duplicate states and yields $\mathscr{H}_j$.

Algorithm 1: Block concatenation and evaluation

Input: State $\mathscr{H} = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$.

Step 1. [Initialization] Set $T_{A_0} \leftarrow 0$, $T_{B_0} \leftarrow 0$, $WC_{\max} \leftarrow 0$.
Step 2. [Evaluation] **for** $i = 1$ **to** K **do**
  $T_{A_i} \leftarrow T_{A_{i-1}} + A_i$
  $T_{B_i} \leftarrow \max\{\, T_{A_{i-1}} + C_i,\ T_{B_{i-1}} + B_i \,\}$
  if $L_i > 0$ **then**
    $WC_{\max} \leftarrow \max\{\, WC_{\max},\ L_i \cdot T_{B_i} \,\}$
  end
end

Output: $WC_{\max}$.

Algorithm 2: DP for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; weights $W_1 > \cdots > W_K$.

Step 1. [Initialization] Set $L_i \leftarrow 0$ for $i = 1, \ldots, K$, set $\mathscr{H}_0 \leftarrow \{(0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0;\ 0, \ldots, 0)\}$ and $Z^* \leftarrow +\infty$.
Step 2. [Generation] **for** $j = 1$ **to** n **do**
  for *each* $(A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K) \in \mathscr{H}_{j-1}$ **do**
    for *each block* $i \in \mathcal{E}(J_j) = \{1, \ldots, i^*\}$ **do**
      $A'_i \leftarrow A_i + a_j$, $B'_i \leftarrow B_i + b_j$, $L'_i \leftarrow w_j$
      $C'_i \leftarrow \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
      Add the state
      $(A_1, \ldots, A'_i, \ldots, A_K; B_1, \ldots, B'_i, \ldots, B_K; C_1, \ldots, C'_i, \ldots, C_K; L_1, \ldots, L'_i, \ldots, L_K)$
      to $\mathscr{H}_j$
    end
  end
end
[Elimination] Elimination $\mathscr{H}_j$ for $j \in \{1, \ldots, n\}$ by Lemma 4.2
Step 3. [Calculate] **for** *each state* $S = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K) \in \mathscr{H}_n$ **do**
  if $L_i \in \{0, W_i\}$ *for every* $i = 1, \ldots, K$ **then**
    $WC_{\max} \leftarrow$ Algorithm 1 applied to S
    $Z^* \leftarrow \min\{\, Z^*,\ WC_{\max} \,\}$
  else
    Discard S.
  end
end
Step 4. [Result] Recover a schedule attaining Z^* by standard backtracking.

Output: An optimal schedule π^*.

To support Step 4, each state in $\mathscr{H}_j$ stores a pointer to the state in $\mathscr{H}_{j-1}$ from which it was derived, together with the block $i \in \mathcal{E}(J_j)$ into which J_j was placed. Starting from a state in $\mathscr{H}_n$ attaining Z^*, backtracking along these pointers recovers the block $\mathcal{B}_i$ containing each job J_j. Sequencing the jobs of each block according to Johnson's rule (Lemma 4.1) and concatenating the blocks then yields an optimal schedule π^* with $WC_{\max}(\pi^*) = Z^*$.

Theorem 4.1. *Algorithm 2 solves* $F2 \parallel WC_{\max}$ *optimally in* $O\big(n \cdot K \cdot |\mathscr{H}|_{\max}\big)$ *time, where* $|\mathscr{H}|_{\max} \le 2^K V^{3K-3}$ *and* $V = \sum_{j=1}^{n} a_j + \sum_{j=1}^{n} b_j$. *For constant* K, *this is* $O(nV^{3K-3})$.

Proof. Each state in $\mathscr{H}_j$ is the $4K$-tuple

$$(A_1, \ldots, A_K;\, B_1, \ldots, B_K;\, C_1, \ldots, C_K;\, L_1, \ldots, L_K),$$

so it suffices to bound the number of distinct values each component can attain.

The quantity A_i is the total M_1 processing time of the jobs assigned to block $\mathcal{B}_i$, hence $A_i \in [0,\, V]$. Since the j jobs considered so far form a prefix of the fixed Johnson sequence, $\sum_{i=1}^{K} A_i$ equals the sum of the a_h over that prefix. At most $K-1$ of the A_i are therefore independent, contributing at most V^{K-1} distinct A-vectors. Symmetrically, $\sum_{i=1}^{K} B_i$ is the prefix sum of the b_j, so the B_i components likewise contribute at most V^{K-1} distinct combinations.

The quantity C_i is the makespan of $\mathcal{B}_i$ when that block is scheduled in isolation by Johnson's rule. Since $C_i \le A_i + B_i \le V$, the C-vector contributes at most V^{K-1} distinct values. The weight L_i equals W_i when block $\mathcal{B}_i$ is non-empty and 0 when it is empty, so the L-vector takes at most 2^K distinct values. Multiplying these bounds gives

$$|\mathscr{H}_j| \le 2^K \cdot V^{K-1} \cdot V^{2K-2} = 2^K \cdot V^{3K-3}.$$

In iteration j, the algorithm takes each state in $\mathscr{H}_{j-1}$, tries every block $i \in \{1, \ldots, K\}$, computes the updated tuple in $O(1)$ time, and inserts it into $\mathcal{Y}_i$. The merge step collects the K sets $\mathcal{Y}_1, \ldots, \mathcal{Y}_K$ and removes duplicates in time proportional to the total number of generated tuples. Iteration j therefore costs $O(K \cdot |\mathscr{H}_{j-1}|)$, and summing over the n iterations gives $O(n \cdot K \cdot |\mathscr{H}|_{\max})$.

The final concatenation by Algorithm 1 evaluates each state in $\mathscr{H}_n$ in $O(K)$ time, which is dominated by the DP phase. For fixed K, 2^K is constant and the bound becomes $O(n \cdot V^{3K-3})$. □

4.3 A 2-approximation

The exact dynamic program of Section 4 runs in pseudo-polynomial time, and the FPTAS of Section 4.4 has running time exponential in the number K of distinct weights. When only a constant-factor guarantee is needed, the following elementary rule suffices.

Algorithm 3: 2-Approximation for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

1 **Step 1.** Sort the jobs by non-increasing weight w_j, breaking ties by non-increasing $a_j + b_j$.

2 **Step 2.** Schedule the jobs in this sequence on both machines.

Output: The resulting permutation schedule π.

Theorem 4.2. *Algorithm 3 runs in $O(n \log n)$ time and returns a schedule with $WC_{\max} \leq 2\,WC^*_{\max}$.*

Proof. Let π be the schedule returned by Algorithm 3. Its jobs appear in non-increasing order of weight, so we name them $J_1, J_2, \ldots, J_n$, with $w_1 \geq w_2 \geq \cdots \geq w_n$ and $\pi = (J_1, J_2, \ldots, J_n)$. Every job of the prefix $S = \{J_1, J_2, \ldots, J_j\}$ then carries a weight of at least w_j.

We first bound the completion times in π. By (1), the job J_j, which sits at position j of π, completes on M_2 at time $C_{\pi(j)}$, and every term of that maximum is bounded by the full sum:

$$\sum_{h=1}^{i} a_h + \sum_{h=i}^{j} b_h \leq \sum_{h=1}^{j} a_h + \sum_{h=1}^{j} b_h = \sum_{h \in S} \big(a_h + b_h\big), \qquad 1 \leq i \leq j.$$

Hence

$$C_{\pi(j)} \;\leq\; \sum_{h \in S} \big(a_h + b_h\big). \tag{26}$$

Since $WC_{\max}(\pi) = \max_{1 \leq j \leq n} w_{\pi(j)}\, C_{\pi(j)}$ ((2)), and $w_{\pi(j)} = w_j$ for the job at position j, multiplying (26) by $w_j > 0$ and taking the maximum over j gives

$$WC_{\max}(\pi) = \max_{1 \leq j \leq n} w_{\pi(j)}\, C_{\pi(j)} \leq \max_{1 \leq j \leq n} w_j \sum_{h \in S} \big(a_h + b_h\big). \tag{27}$$

It remains to bound the right-hand side by $2WC^*_{\max}$. Fix a prefix $S = \{J_1, J_2, \ldots, J_j\}$ and let π^* be an optimal schedule. Three quantities bound the optimum from below,

$$LB_1 = \max_{1 \leq j \leq n} w_j \sum_{h \in S} a_h, \tag{28}$$

$$LB_2 = \max_{1 \leq j \leq n} w_j \sum_{h \in S} b_h, \tag{29}$$

$$LB_3 = \max_{1 \leq j \leq n} w_j\,(a_j + b_j), \tag{30}$$

the M_1 threshold LB_1, the M_2 threshold LB_2 and the job threshold LB_3, and their largest is the reference value of the instance,

$$LB = \max\{LB_1, LB_2, LB_3\}, \tag{31}$$

so that $LB \leq WC^*_{\max}$.

To see this, let J_u be the job of S that finishes last on M_1 in π^*, and J_v the job of S that finishes last on M_2. All jobs of S are processed on M_1 before J_u completes M_1, so $C^*_u \geq \sum_{h\in S} a_h$. In the same way, all M_2 processing of the jobs of S precedes the completion of J_v on M_2, so $C^*_v \geq \sum_{h\in S} b_h$. Since both J_u and J_v lie in S, their weights satisfy $w_u, w_v \geq w_j$. The value $WC^*_{\max}$ bounds the weighted completion time of every job of π^*, so $WC^*_{\max} \geq w_u C^*_u \geq w_j \sum_{h\in S} a_h$ and $WC^*_{\max} \geq w_v C^*_v \geq w_j \sum_{h\in S} b_h$. Every job J_j of S is processed on M_1 before it is processed on M_2, and neither operation can be left out, so $C^*_j \geq a_j + b_j$ and $WC^*_{\max} \geq w_j C^*_j \geq w_j(a_j + b_j)$. The prefix S was arbitrary, so these bounds hold for every prefix.

For the prefix S, the first two bounds give $w_j \sum_{h\in S}(a_h + b_h) \leq LB_1 + LB_2 \leq 2\,LB \leq 2\,WC^*_{\max}$, and the claim follows from (27). □

4.4 A fully polynomial-time approximation scheme

4.4.1 Rounding the time coordinates

The exact dynamic program of Algorithm 2 becomes a fully polynomial-time approximation scheme once the input data are rounded (Hall, 1994). The technique rests on bounds for the coordinates that are rounded: each time coordinate ranges over a bounded interval, and rounding it to one of a fixed number of geometrically spaced values keeps the relative error within a factor θ. The total processing times A_i, B_i of block $\mathcal{B}_i$ on M_1 and M_2, and its completion time C_i, are rounded. The weight L_i of the last job of block $\mathcal{B}_i$ is not rounded; following Algorithm 2, the positional constraint $L_i \in \{0, W_i\}$ is required for every block, and a state violating it is discarded.

Fix $0 < \epsilon < 1$ and let

$$\theta = 1 + \frac{\epsilon}{2n}, \qquad V_A = \sum_{j=1}^{n} a_j, \qquad V_B = \sum_{j=1}^{n} b_j, \qquad V = V_A + V_B.$$

For every block $\mathcal{B}_i$, the recurrence (25) preserves $A_i \leq V_A$ and $B_i \leq V_B$, since block $\mathcal{B}_i$ contains a subset of the jobs; moreover $C_i \leq A_i + B_i \leq V$. We therefore round the A dimension over $[0, V_A]$, the B dimension over $[0, V_B]$, and the C dimension over $[0, V]$. To this end, let

$$\Delta(x) = k = \lfloor \log_\theta x \rfloor \quad \text{for every } x \geq 1 \text{ with } \theta^k \leq x < \theta^{k+1}, \qquad \text{and} \qquad \Delta(0) = -1.$$

All processing times are integral, so every value taken by a coordinate A_i, B_i, C_i is a non-negative integer. Hence any two values $y, y' \geq 1$ with $\Delta(y) = \Delta(y')$ satisfy $y \leq \theta\, y'$; we call this the *interval-dominance* property, and it is the only fact about the rounding used in the analysis below.

The *label* of a state $\mathscr{H}^\epsilon = (A_1, \ldots, A_K; B_1, \ldots, B_K; C_1, \ldots, C_K; L_1, \ldots, L_K)$ is the tuple

formed by applying Δ to its time coordinates, together with its block-end weights,

$$\ell(\mathscr{H}^\epsilon) = \big(\Delta(A_1),\dots,\Delta(A_K);\ \Delta(B_1),\dots,\Delta(B_K);\ \Delta(C_1),\dots,\Delta(C_K);\ L_1,\dots,L_K\big).$$

Thus two states share a label exactly when every coordinate A_i, B_i and C_i falls into the same box and their block-end weights coincide. Each dimension $x \in \{A, B, C\}$ contributes at most $v^x + 2$ distinct values of Δ, where $v^x = \lceil \log_\theta V_x \rceil$ and $V_C = V$, while the weight is not rounded and each L_i takes at most $n + 1$ values.

The Elimination step keeps one representative per label. At iteration j, every state of $\mathscr{H}^\epsilon_{j-1}$ spawns one candidate for each eligible block $i \in \mathcal{E}(J_j) = \{1,\dots,i^*\}$, and each candidate is assigned its label. Two candidates with the same label are indistinguishable after rounding and carry the same block-end weights $L_1,\dots,L_K$, so the step keeps any one of them and discards the rest: the kept state satisfies $x \le \theta\,\tilde{x}$ in every time coordinate $x \in \{A, B, C\}$ of every block, relative to every candidate $\tilde{\mathscr{H}}^\epsilon$ it replaces, while its block-end weights are unchanged. Every label therefore carries exactly one state into $\mathscr{H}^\epsilon_j$, and the number of states never exceeds the number of labels.

Algorithm 4: State-class FPTAS for $F2 \,\|\, WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j); block count K; accuracy ϵ.

Step 1. [Initialization] Set $\theta = 1 + \epsilon/(2n)$; set $Z^* \leftarrow +\infty$;
$\mathscr{H}^\epsilon_0 \leftarrow \{(0,\dots,0;\ 0,\dots,0;\ 0,\dots,0;\ 0,\dots,0)\}$.
Step 2. [Generation] **for** $j = 1$ **to** n **do**
  for *each state* $(A_1,\dots,A_K;B_1,\dots,B_K;C_1,\dots,C_K;L_1,\dots,L_K) \in \mathscr{H}^\epsilon_{j-1}$ **do**
    for *each block* $i \in \mathcal{E}(J_j) = \{1,\dots,i^*\}$ **do**
      $\tilde{A}_i = A_i + a_j$, $\tilde{B}_i = B_i + b_j$, $\tilde{L}_i = w_j$
      $\tilde{C}_i = \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
      [Label] Record the label $\ell(\tilde{\mathscr{H}}^\epsilon_j) =$
        $\big(\Delta(\tilde{A}_1),\dots,\Delta(\tilde{A}_K);\ \Delta(\tilde{B}_1),\dots,\Delta(\tilde{B}_K);\ \Delta(\tilde{C}_1),\dots,\Delta(\tilde{C}_K);\ \tilde{L}_1,\dots,\tilde{L}_K\big)$.
    end
    [Elimination] Among the candidate states sharing the same label, keep one of them and delete the others; set $\mathscr{H}^\epsilon_j$ to the kept states.
  end
end
Step 3. [Calculate] Same as Step 3 of Algorithm 2, applied to $\mathscr{H}^\epsilon_n$.
Step 4. [Result] Same as Step 4 of Algorithm 2.
Output: An approximate schedule π.

Lemma 4.3. *Let $\mathscr{H}_j = (A_1,\dots,A_K;B_1,\dots,B_K;C_1,\dots,C_K;L_1,\dots,L_K)$, $j = 1,\dots,n$, be the states along an optimal trajectory of Algorithm 2, the states that backtracking from an optimal final state recovers. For every j, Algorithm 4 keeps a state $\hat{\mathscr{H}}^\epsilon_j = (\hat{A}_1,\dots,\hat{A}_K;\hat{B}_1,\dots,\hat{B}_K;\hat{C}_1,\dots,\hat{C}_K;L_1,\dots,L_K)$ that carries the same block-end weights $L_1,\dots,L_K$ and satisfies*

$$\hat{A}_i \le A_i\,\theta^{j-1},\quad \hat{B}_i \le B_i\,\theta^{j-1},\quad \hat{C}_i \le C_i\,\theta^{j-1},\quad (1 \le i \le K). \tag{32}$$

Proof. We argue by induction on j, from 1 to n.

Base case $j = 1$. Both algorithms generate their candidates for J_1 from the same zero state. Two candidates that place J_1 in different blocks differ in at least one time coordinate, so they carry different labels and no state is deleted. The sets $\mathscr{H}_1^\epsilon$ and $\mathscr{H}_1$ therefore coincide, and (32) holds with equality at $j = 1$.

Inductive step $j \geq 2$. Assume that (32) holds at $j-1$, and let i be the block to which $\mathscr{H}_{j-1}$ appends J_j in Algorithm 2. Algorithm 2 assigns a job only to a block i with $w \leq W_i$, so $w_j \leq W_i$. This constraint depends only on J_j and the block index, hence appending J_j to block i is available from $\hat{\mathscr{H}}_{j-1}^\epsilon$ as well. Write A_i', B_i' and C_i' for the coordinates of $\mathscr{H}_{j-1}$. Applying the exact recurrences to $\hat{\mathscr{H}}_{j-1}^\epsilon$ and using $\theta^{j-2} \geq 1$ gives a candidate $\tilde{\mathscr{H}}_j^\epsilon$ with

$$\begin{aligned}
\tilde{A}_i &= \hat{A}_i + a_j \leq \big(A_i' + a_j\big)\theta^{j-2} = A_i\theta^{j-2},\\
\tilde{B}_i &= \hat{B}_i + b_j \leq \big(B_i' + b_j\big)\theta^{j-2} = B_i\theta^{j-2},\\
\tilde{C}_i &= \max\big\{\hat{C}_i + b_j,\ \hat{A}_i + a_j + b_j\big\}\\
&\leq \max\big\{C_i' + b_j,\ A_i' + a_j + b_j\big\}\theta^{j-2} = C_i\theta^{j-2},
\end{aligned}$$

where A_i, B_i and C_i are the coordinates of $\mathscr{H}_j$. For the remaining blocks $m \neq i$ the same transition leaves the coordinates unchanged, so that $\tilde{x}_m = \hat{x}_m \leq x_m'\theta^{j-2} = x_m\theta^{j-2}$ for $x \in \{A, B, C\}$. Hence $\tilde{x}_m \leq x_m\theta^{j-2}$ holds for every block $m = 1, \ldots, K$. The block-end weights $L_1, \ldots, L_K$ of the candidate are those of $\mathscr{H}_j$, because $L_i = w_j$ and the remaining block-end weights are unchanged.

The candidate $\tilde{\mathscr{H}}_j^\epsilon$ carries the label $\ell(\tilde{\mathscr{H}}_j^\epsilon)$ assigned in Algorithm 4, and the algorithm keeps a state $\hat{\mathscr{H}}_j^\epsilon \in \mathscr{H}_j^\epsilon$ with that same label. Since the label records the block-end weights, $\hat{\mathscr{H}}_j^\epsilon$ carries $L_1, \ldots, L_K$ as well. Interval dominance states that two values with the same index Δ differ by at most a factor θ, so $y_m^x \leq \theta\,\tilde{x}_m$ for every $x \in \{A, B, C\}$ and every block $m = 1, \ldots, K$, where y_m^x denotes the coordinates of $\hat{\mathscr{H}}_j^\epsilon$. Together with $\tilde{x}_m \leq x_m\theta^{j-2}$, this yields (32) at j and completes the induction.

Every state of $\mathscr{H}_n^\epsilon$ stores all $3K$ time coordinates, so the induction covers the coordinates of the last block $\mathcal{B}_K$ as well. Storing these coordinates for every block, exactly as in Algorithm 2, rather than deriving them from the others, preserves the factor θ^{j-1} of (32): a derived value would accumulate the rounding errors of the remaining blocks. □

Theorem 4.3. *For every $0 < \epsilon < 1$, Algorithm 4 returns a schedule with*

$$WC_{\max} \;\leq\; (1+\epsilon)\,WC_{\max}^*.$$

For fixed K, it runs in $O\Big(n^{4K+1} \cdot K \cdot \frac{(\log V)^{3K}}{\epsilon^{3K}}\Big)$ time, which is polynomial in n, $\log V$ and $1/\epsilon$.

Proof. We first bound the value of the returned schedule, and then its running time.

Let $\mathscr{H}_n$ be the final state of the optimal trajectory of Algorithm 2, and let $\hat{\mathscr{H}}_n^{\epsilon} \in \mathscr{H}_n^{\epsilon}$ be the state guaranteed by Lemma 4.3 at $j = n$. By the lemma, the time coordinates of $\hat{\mathscr{H}}_n^{\epsilon}$ are within a factor θ^{n-1} of those of $\mathscr{H}_n$, and its block-end weights are those of $\mathscr{H}_n$, because the weights are not rounded. The final state $\mathscr{H}_n$ satisfies the positional constraint $L_i \in \{0, W_i\}$ of Algorithm 2, so every block of $\hat{\mathscr{H}}_n^{\epsilon}$ has block-end weight at most W_i.

Let $\hat{\pi}$ and π^* be the schedules obtained by concatenating the blocks of $\hat{\mathscr{H}}_n^{\epsilon}$ and $\mathscr{H}_n$, and let $\hat{C}_i$ and C_i denote the completion times of the last job of block $\mathcal{B}_i$ in the two schedules, which are the corresponding coordinates of the two states. By the concatenation recurrence of Algorithm 1, C_i is nondecreasing in every coordinate of the state and positively homogeneous. Hence $\hat{C}_i \le \theta^{n-1} C_i$ for every i. Since every block of $\hat{\mathscr{H}}_n^{\epsilon}$ has block-end weight at most W_i, Lemma 4.1 yields

$$WC_{\max}(\hat{\pi}) = \max_{1 \le i \le K} W_i \hat{C}_i \le \theta^{n-1} \max_{1 \le i \le K} W_i C_i = \theta^{n-1} WC^*_{\max}.$$

The factor θ^{n-1} is at most $1 + \epsilon$ for $0 < \epsilon < 1$. Because $1 + x \le e^x$ for every real x,

$$\theta^{n-1} = \left(1 + \frac{\epsilon}{2n}\right)^{n-1} \le e^{\frac{(n-1)\epsilon}{2n}} < e^{\epsilon/2},$$

where the strict inequality uses $(n-1)/(2n) < 1/2$. It is therefore enough to check that $e^{\epsilon/2} \le 1 + \epsilon$ on $[0, 1]$. The function $g(\epsilon) = 1 + \epsilon - e^{\epsilon/2}$ satisfies $g(0) = 0$ and

$$g'(\epsilon) = 1 - \tfrac{1}{2} e^{\epsilon/2} \ge 1 - \tfrac{1}{2} e^{1/2} > 0 \qquad (0 \le \epsilon \le 1),$$

because $e^{\epsilon/2} \le e^{1/2} < 2$ on that interval. Hence g is increasing, so $g(\epsilon) \ge 0$ and $e^{\epsilon/2} \le 1 + \epsilon$. Consequently $\theta^{n-1} \le 1 + \epsilon$, and the schedule returned by Algorithm 4, being no worse than $\hat{\pi}$, has a value of at most $(1 + \epsilon)\, WC^*_{\max}$.

We now bound the running time. At each level j, Algorithm 4 keeps at most one state per label, and the number of labels is $(v^A + 2)^K (v^B + 2)^K (v^C + 2)^K (n + 1)^K$. The rounded time coordinates contribute $(v^A{+}2)^K (v^B{+}2)^K (v^C{+}2)^K$, which is at most $(v^C{+}2)^{3K}$ since $v^A, v^B \le v^C$, and each of the K block-end weights takes at most $n + 1$ values. Each state generates at most K candidates, one per eligible block, in $O(1)$ time each. The final evaluation of the states of $\mathscr{H}^{\epsilon,n}$ costs $O(K)$ per state. The running time is therefore $O\big(nK\,(v^C{+}2)^{3K}(n{+}1)^K\big)$. Since $\ln(1 + \epsilon/(2n)) \ge \epsilon/(4n)$ for $0 < \epsilon < 1$, we have

$$v^C = \left\lceil \frac{\ln V}{\ln \theta} \right\rceil = O\left(\frac{n \log V}{\epsilon}\right).$$

For constant K, the bound reduces to $O(n^{4K+1} K (\log V)^{3K} / \epsilon^{3K})$, which is polynomial in n, $\log V$ and $1/\epsilon$. □

4.4.2 Rounding the weights

The FPTAS above rounds the time coordinates A_i, B_i, C_i and assumes that the number K of distinct weights is fixed. An analogous geometric rounding applies to the weights

themselves when their ratio is bounded. Let $W_{\min} = \min_j w_j$, $W_{\max} = \max_j w_j$, and suppose $W_{\max}/W_{\min} \le c$ for a constant c. Partition the interval $[W_{\min}, W_{\max}]$ into

$$J_0 = [W_{\min}, W_{\min}\theta), \qquad J_r = [W_{\min}\theta^r, W_{\min}\theta^{r+1}) \ \ (1 \le r \le q-1), \qquad J_q = [W_{\min}\theta^q, W_{\max}],$$

with $q = \lceil \log_\theta(W_{\max}/W_{\min}) \rceil$, and round each weight w_j upward to the right endpoint of its interval,

$$\hat{w}_j = W_{\min}\, \theta^{\,r_j+1}, \qquad r_j = \left\lfloor \log_\theta \frac{w_j}{W_{\min}} \right\rfloor,$$

so that $w_j \le \hat{w}_j \le \theta\, w_j$. The rounded weights take only $q+1 = O\big(\log_\theta(W_{\max}/W_{\min})\big)$ distinct values, so jobs whose weights differ by a factor at most θ merge into a single class.

Lemma 4.4. *Let $\hat{w}_j$ be the rounded weights defined above, and let $\hat{\pi}$ be an optimal schedule of the instance with weights $\hat{w}_j$, whose optimal value is $\hat{W}C^*_{\max}$. Then*

$$WC_{\max}(\hat{\pi}) \le \theta\, WC^*_{\max}. \tag{33}$$

Proof. By construction $w_j \le \hat{w}_j \le \theta w_j$ for every job J_j. By (2) and the monotonicity of the maximum, every schedule π satisfies

$$WC_{\max}(\pi) = \max_{1\le j\le n} w_{\pi(j)} C_{\pi(j)} \le \max_{1\le j\le n} \hat{w}_{\pi(j)} C_{\pi(j)} = \hat{W}C_{\max}(\pi) \le \theta\, WC_{\max}(\pi).$$

Let π^* be an optimal schedule of the original instance. The rounded problem is solved optimally by $\hat{\pi}$, and π^* is feasible for it, so

$$WC_{\max}(\hat{\pi}) \le \hat{W}C_{\max}(\hat{\pi}) = \hat{W}C^*_{\max} \le \hat{W}C_{\max}(\pi^*) \le \theta\, WC_{\max}(\pi^*) = \theta\, WC^*_{\max},$$

which is (33). □

Hence the weight rounding incurs a relative error of at most $\theta = 1 + \epsilon/(2n) \le 1 + \epsilon$, exactly as in the completion-time rounding.

5 Heuristic algorithms

The exact dynamic program of Section 4 is pseudo-polynomial, and the running time of the FPTAS of Section 4.4 is exponential in the number K of distinct weights. Fast heuristics are therefore the practical route for large instances. This section presents two of them for $F2 \parallel WC_{\max}$: an adaptation of the NEH algorithm (Puka et al., 2024) and a genetic algorithm (Qiao et al., 2022).

Throughout this section, the objective of a permutation $\pi = (\pi(1), \ldots, \pi(k))$ of k jobs is evaluated by (1) and (2), so that

$$WC_{\max}(\pi) = \max_{1\le t\le k} w_{\pi(t)}\, C_{\pi(t)},$$

which is computed in $O(k)$ time.

5.1 The NEH heuristic

Constructive heuristics build a complete schedule in a single pass and need no tuning, and NEH (Puka et al., 2024) is among the most effective of them for permutation flow shops: the jobs are inserted one at a time into a growing partial sequence, each at the position that best serves the objective.

Two properties of the objective shape the adaptation. A job with a large $a_j + b_j$ delays every job behind it on both machines and should therefore come early, and among jobs of similar length a larger weight w_j gains more from an early position. The jobs are therefore sorted by non-increasing $a_j + b_j$, ties broken by non-increasing w_j, and each is inserted where $WC_{\max}$ is smallest; Algorithm 5 gives the procedure.

Evaluating the k insertions of step k costs $O(k^2)$ by (1), so the run takes $O(n^3)$ time, as for the original algorithm (Nawaz et al., 1983). The heuristic returns a permutation schedule by construction and needs no hyperparameter, which makes it the fast standalone solver for large instances.

Algorithm 5: NEH heuristic for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j).

Step 1. [Sequence] Sort the jobs by non-increasing $a_j + b_j$, breaking ties by w_j; let $J_1, \ldots, J_n$ be the resulting sequence.
Step 2. [Initialization] $\pi \leftarrow (J_1)$.
Step 3. [Insertion] **for** $k = 2$ **to** n **do**
  $\pi \leftarrow$ the sequence obtained by inserting J_k into π at a position minimizing $WC_{\max}$.
end

Output: Schedule π.

5.2 The genetic algorithm

The objective is a maximum: one *critical* job attains it, and its weighted completion time is the value of the whole schedule. An improvement must therefore lower the completion time of that job, and every other weighted completion time must stay below the new maximum. The two aims this sets pull in opposite directions. Placing heavy jobs early lowers their weighted completion times but pushes the light ones back and lengthens the makespan, while sequencing on the processing times alone shortens the makespan and ignores the weights. A genetic algorithm for $F2 \parallel WC_{\max}$ therefore cannot treat the jobs symmetrically.

We use the genetic algorithm in the framework that Qiao et al. (2022) propose for flow-shop scheduling: a permutation chromosome, an initial population of Johnson's schedule and $N - 1$ neighbours of it, binary tournament selection, partially mapped crossover, and, for survival, the non-dominated sorting and crowding distance of NSGA-II (Deb et al., 2002), which recent flow-shop studies extend (Wang et al., 2023). The survival ranks the pooled

parents and offspring on the makespan, the weight of the closing job and the objective. The operators are those of that framework; what is adapted here is the mutation, which moves blocks of one weight rather than single jobs, so that the search follows the structure of Lemma 4.1. The run closes with a local search over the k jobs that carry the largest weighted completion times, each reinserted at the position that lowers the objective most. Algorithms 6 to 8 give the operators and the algorithm.

5.2.1 Encoding and decoding

Encoding. A chromosome is a permutation $\pi = (\pi(1), \ldots, \pi(n))$ of the n jobs, in which the gene at position t is the job scheduled t-th. Its length is the number of jobs and its alphabet the job set $\mathcal{J}$, and since both machines process the jobs in the order the chromosome gives, every chromosome stands for a permutation schedule and every permutation schedule for exactly one chromosome. No repair operator is therefore needed to restore feasibility after crossover or mutation.

Decoding. Both machines process the jobs in the order $\pi(1), \ldots, \pi(n)$. The completion time $C_{\pi(t)}$ of the t-th job follows from the closed form (1), which is evaluated in $O(n)$ time using the prefix sums $A_t = \sum_{h=1}^{t} a_{\pi(h)}$ and $B_t = \sum_{h=1}^{t} b_{\pi(h)}$; the objective value is that of (2),

$$WC_{\max}(\pi) = \max_{1 \le t \le n} w_{\pi(t)}\, C_{\pi(t)}.$$

The fitness of a schedule is the objective itself, $WC_{\max}(\pi)$, and no normalised scale is fixed for it. Figure 4 shows the encoding and the decoding on the six-job instance of Example 4.1.

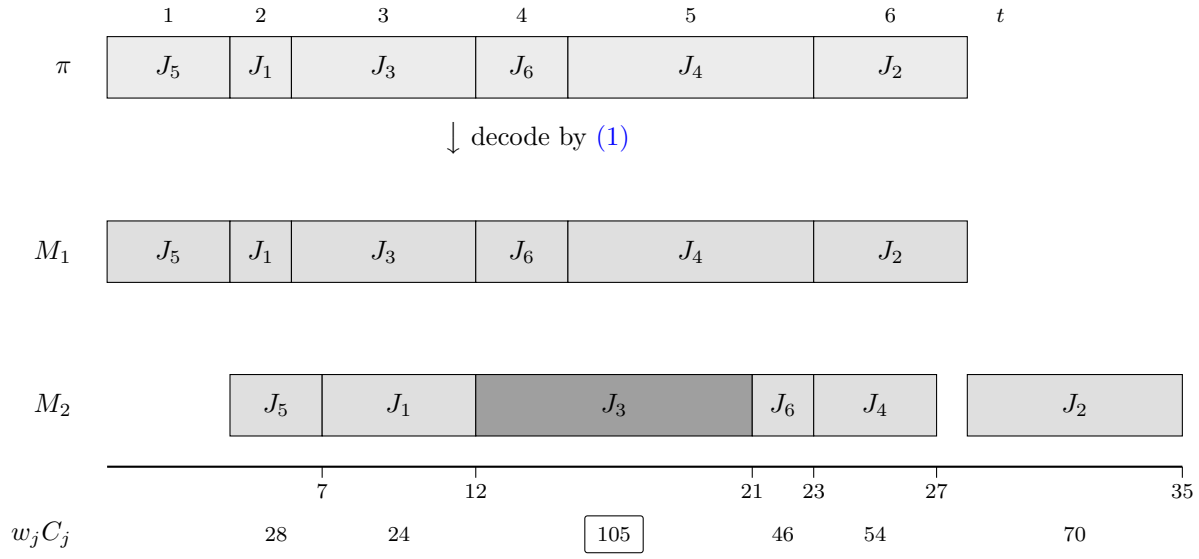


Figure 4: Encoding and decoding

5.2.2 Crossover and mutation

The crossover is the partially mapped crossover (PMX), the operator the framework uses for permutation chromosomes. Two cut points $p_1 < p_2$ are drawn, the segment $\pi_1[p_1..p_2]$ is copied into the child, and the remaining positions are filled from π_2 through the mapping that the segment induces between the two parents, so that every position receives exactly one job and the child is always a valid permutation. Figure 5 shows one application: the grey cells are the copied segment, the three double arrows are the mapping it induces, and the red cells are the chain followed at position 1 of the first child, where the gene taken from π_2 already occurs in the segment. Algorithm 6 gives the operator.

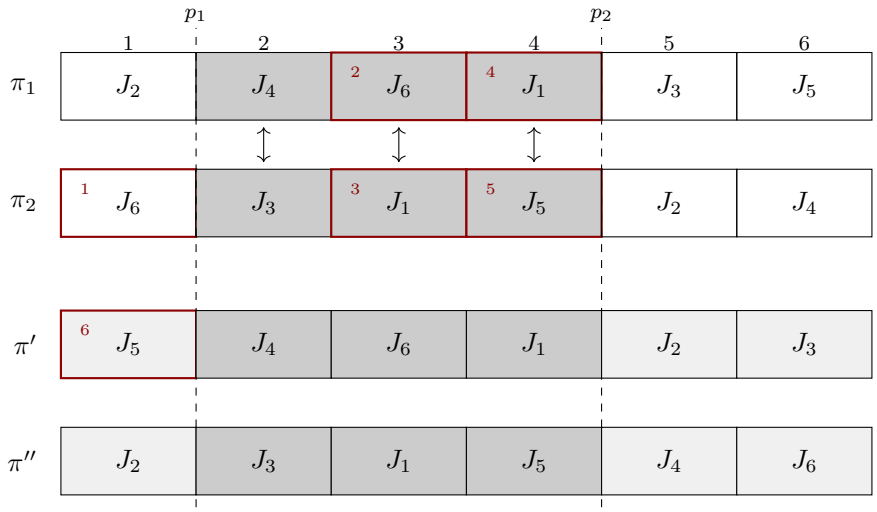


Figure 5: One application of the partially mapped crossover, with $p_1 = 2$ and $p_2 = 4$

Algorithm 6: Partially Mapped Crossover (PMX) for $F2 \parallel WC_{\max}$

Input: Parents π_1, π_2; cut points $p_1 < p_2$.

Step 1. [Copy segment] Set $\pi'[t] \leftarrow \pi_1[t]$ for $t = p_1, \ldots, p_2$.
Step 2. [Fill outer positions] **for** $t = 1$ **to** n, $t \notin [p_1, p_2]$ **do**
  $j \leftarrow \pi_2[t]$
  while j *occurs in* π' **do**
    $j \leftarrow \pi_2[\pi_1^{-1}(j)]$ `// follow the PMX mapping`
  end
  $\pi'[t] \leftarrow j$
end

Output: Child π'.

The crossover reads nothing of the weights, so the mutation is what keeps the search moving across them. Rather than one job at a time, it removes the run of jobs that share one weight, keeping their order, and reinserts the run at the position that minimizes $WC_{\max}(\pi)$; the weight is that of a job attaining the maximum, or, with probability one half, one drawn at random, so that the order of the blocks is explored as well as repaired. Moving a block

forward delays the jobs it passes, so the move is proposed whether or not it lowers the objective, and it is the survival of the generation that decides what becomes of it. Algorithm 7 gives the operator.

Algorithm 7: Block Mutation for $F2 \parallel WC_{\max}$

Input: Schedule π; weights w_j.

Step 1. [Block] Evaluate $WC_{\max}(\pi)$ by (1) and let j^* be a job attaining the maximum; with probability $1/2$ let $W \leftarrow w_{j^*}$, otherwise let W be a weight drawn at random from the instance.

Step 2. [Move] Form the run of the jobs of weight W, keeping the order π holds them in, and remove it from π.

for *every position r of the remaining schedule* **do**

  Form $\pi^{(r)}$ by inserting the run at position r and evaluate $WC_{\max}(\pi^{(r)})$.

end

$\pi \leftarrow \pi^{(r^*)}$, where $r^* = \arg\min_r WC_{\max}(\pi^{(r)})$, whichever way the objective moves.

Output: Schedule π.

Algorithm 8: Genetic Algorithm for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$; population size N; generations G; offspring λ; probabilities p_c, p_m; reinsertion count k.

Step 1. [Initialization] Build P_0 from Johnson's schedule and $N-1$ neighbours of it; evaluate all individuals; set π^* to the best.
Step 2. [Evolution] **for** $g = 1$ **to** G **do**
  Selection Form the mating pool M_g from P_{g-1} by binary tournament selection.
  Crossover while $|O_g| < \lambda$ **do**
    Draw two parents from M_g; with probability p_c apply Algorithm 6 to obtain two children, otherwise copy the parents into the offspring set O_g.
  end
  Mutation Apply Algorithm 7 to each offspring with probability p_m; evaluate every child by (1).
  Survival $P_g \leftarrow$ the N survivors of $P_{g-1} \cup O_g$, taken by non-dominated sorting on the three objectives and, on the last front, by crowding distance.
  Elite If a child of O_g improves on $WC_{\max}(\pi^*)$, replace π^* by it, and if π^* is not among the survivors, let it take the place of the one the ranking leaves last, so that it is a parent of generation $g+1$.
end
Step 3. [Local search] **repeat**
  Let $J_1, \ldots, J_k$ be the jobs of π^* with the largest $w_j C_j$.
  Form, for every J_i and every position r, the schedule obtained by moving J_i to position r, and evaluate it by (1); let π' be the best of them.
  if $WC_{\max}(\pi') < WC_{\max}(\pi^*)$ **then**
    $\pi^* \leftarrow \pi'$
  end
until *no reinsertion of the step improves*

Output: Best schedule π^*.

6 Experiments

All methods were coded in Python 3.13 and run on a machine with an Intel Core Ultra 5 230F processor and 16 GB of memory, under Windows 11. The MILP model was solved with Gurobi 13.0 on a single thread, so that the reported times are comparable across configurations. Every method is run once on every instance and given the same 600 s budget, and all of them evaluate the objective by the same recurrence (1), so that the comparison holds the objective fixed and varies only the search. Two readings are taken. The dynamic program, the only exact method of the study, is read on the number of instances whose optimum it establishes and on its running time. Every other method is read on its relative

percentage gap

$$\mathrm{gap}_A = \frac{z_A - z_{\mathrm{base}}}{z_{\mathrm{base}}}, \tag{34}$$

where z_A is the objective value of the schedule it returned and z_{base} the reference value of the instance, and on the wall-clock time it requires; the gap is reported in per cent and the time in milliseconds, each by its mean and its maximum over the instances of the configuration. On the small scale the reference is the optimum, which is the value the model returns, so that the model is read on a gap of zero and every heuristic on its distance from the optimum. On the large scale, where no exact method is within reach, the reference is the lower bound LB of (31), the largest of the three thresholds LB_1, LB_2 and LB_3 of Section 4.3, each of which the instance determines on its own, so that no schedule of a method enters the denominator of its gap.

The instances were generated in three groups, every integer of an instance drawn from a normal distribution on the integers of its range: the mean at the middle, the standard deviation a sixth of the width, the draw rounded and clamped. The first group, on which the dynamic program is reported, crosses the sizes 40, 60, 80 and 100 with the weight count $K = 2$, whose weights are the geometric ladder $1, 10$, and with the two ranges $[1, 40]$ and $[1, 80]$. Its ladder rises by 20 a step and stops where the time limit binds: at $[1, 40]$ every instance is settled well within it, while on $[1, 80]$ six of the twenty at a hundred jobs are cut short. This is not the set on which the methods are compared: the running time $O(nK2^K V^{3K-3})$ grows so fast in the number K of distinct weights that the sizes the other algorithms are read at are out of its reach, so the group is reported on a set of its own. The two groups that follow are the comparison, and both carry the two ranges of the first. Their instances hold at most thirty distinct weights, drawn by the same rule on $\{1, \ldots, 30\}$, so that the classes are neighbouring integers rather than the decades of the first group; their processing times are drawn the same way, independently for every job, on one of $[1, 40]$ and $[1, 80]$, named in the caption of every table. The small group varies the instance size over $10, 15, \ldots, 60$ and the large one over $100, 120, \ldots, 400$: the model is compared with the heuristics at the small sizes, where it is exact and settles them, and the heuristics alone at the large ones, where the gap of each is measured against LB of (31). Every configuration is replicated over twenty independent instances and every figure below is their average, and a method that reaches the limit is recorded with the value it has attained. Every table reads the same way: a row is an instance size, the columns are grouped by method (by range in the table of the dynamic program), and every quantity is read from two columns, its mean under "avg" and its maximum under "max"; gaps are in per cent, running times in milliseconds in the form 1.5e3 for 1.5×10^3, and "opt." counts the instances the dynamic program settles. The smallest mean gap of a row is set in bold, judged on the unrounded values. For brevity, *the model* is the MILP formulation, *the program* the dynamic program, *the rules* the 2-approximation and NEH together, and *the search* the genetic algorithm. The section answers four questions in turn: how far the dynamic program reaches, how close the

two rules come and at what cost, when the search overtakes them and at what price, and how much room the lower bound leaves in the gaps of the large scale.

6.1 The dynamic program

The dynamic program is the only method of the study that certifies an optimum, and its strength is that it is exact wherever it returns: seven of the eight configurations settle all twenty of their instances, and a forty-job instance costs 1.47 s on $[1, 40]$ and 4.98 s on $[1, 80]$ (Table 1). Its weakness is reach, and it is bounded from two sides. The first is the number of weight classes. Its state space is bounded by $2^K V^{3K-3}$, with V the total processing time of the instance (Section 4), so that a third class multiplies the number of states by $2V^3$: at $K = 3$ the memory of the machine the runs were made on is exhausted and the program does not return, which is why the group is reported at $K = 2$ alone. The second is the instance size and the width of the range, which enter through V as well. At $a_j, b_j \in [1, 40]$ the mean running time rises from 1.47 s at forty jobs to 14.23 s at sixty, 45.13 s at eighty and 89.59 s at a hundred – a factor of 61 over the ladder, against an objective that grows only from 4335.0 to 10647.0, a factor of 2.5. The range multiplies that time by between 2.9 and 3.7 at every size, the two extremes being 1.47 against 4.98 s at forty jobs and 89.59 against 329.01 s at a hundred. The two bounds meet at the top of the ladder: at a hundred jobs on $[1, 80]$ the 600 s limit cuts six of the twenty instances short, so that the configuration settles 14/20, and over the fourteen it settles the mean time is 329.01 s with a largest of 628.50 s, past the limit because the deadline is checked once per job and not inside one. The exact method is therefore cheap at forty jobs and out of reach beyond a hundred, while the heuristics are read to four hundred.

Table 1: The dynamic program, $K = 2$

n	$a_j, b_j \in [1, 40]$					$a_j, b_j \in [1, 80]$				
	$WC_{\max}$		time (ms)		opt.	$WC_{\max}$		time (ms)		opt.
	avg	max	avg	max		avg	max	avg	max	
40	4335.0	5480.0	1.5e3	4.3e3	20/20	8604.5	10590.0	5.0e3	2.4e4	20/20
60	6340.0	7270.0	1.4e4	3.1e4	20/20	12344.5	15020.0	5.3e4	1.1e5	20/20
80	8221.0	9750.0	4.5e4	1.1e5	20/20	17341.5	19580.0	1.3e5	2.8e5	20/20
100	10647.0	12350.0	9.0e4	1.7e5	20/20	21544.3	24240.0	3.3e5	6.3e5	14/20

Averages at $n = 100$ on $[1, 80]$ cover the 14 instances settled within the 600 s limit; the other six are cut short.

6.2 The small scale

The model settles every instance of the small scale to optimality – 20 of 20 in each of its 22 configurations, 440 instances in all – so the gaps reported against it are distances to an

optimum and not to the incumbent of a run that ran out of time. What grows is its cost: the mean time of an instance rises from 50.6 ms at ten jobs to 80.1 s at sixty on $[1, 40]$, and from 40.2 ms to 43.5 s on $[1, 80]$. The cost is not monotone in the range: at four of the eleven sizes, sixty jobs among them, the wider set is the cheaper of the two, so the time is read against the range rather than averaged over both. The value rises with the range as well – the mean optimum of a ten-job instance is 2510.7 on $[1, 40]$ and 5433.0 on $[1, 80]$, and that of a sixty-job instance 12323.6 and 25266.1. The slowest instance of the scale takes 299.6 s of the 600 s a run is allowed, so the exact side is complete here but not roomy: the limit is close by the top of the scale.

The two rules that read an instance without searching it are close to the optimum and far apart in what they cost. The 2-approximation, which reads the weights alone, has a mean gap of 3.52 per cent over the whole small scale – 3.47 on $[1, 40]$ and 3.56 on $[1, 80]$ – and reaches the optimum on 36 of the 440 instances, its worst 16.71 per cent away. NEH has a mean gap of 2.53 per cent, 2.39 and 2.67 on the two ranges, reaches the optimum on 99 of the 440, and its worst instance is 11.27 per cent away. The two reverse over the scale: NEH is the closer at nine of the eleven sizes on $[1, 40]$ and at eight on $[1, 80]$, and the 2-approximation takes the lead at the largest sizes, where the order of the weights is what the objective rewards. Their costs are not of the same order either: 0.02 ms an instance on average for the 2-approximation, never above 0.05 ms, against 2.68 ms for NEH, which grows with the instance to 8 ms at sixty jobs; the model is 5,200 times above NEH and some 7×10^5 times above the 2-approximation, at 13.85 s an instance on the average of the scale.

The genetic algorithm starts furthest from the optimum and closes the distance over the small scale, which is its strength here. Its mean gap falls from 16.83 to 3.92 per cent on $[1, 40]$ and from 13.16 to 3.24 per cent on $[1, 80]$, which leaves it within a point of both rules at sixty jobs. What the rest of the scale shows is what that approach costs. It starts as the worst of the three, at three to twenty times the gap of the rules at ten jobs. Its course is not monotone: it climbs from 15 per cent at ten jobs to 19 at fifteen, fluctuates between 9 and 16 through forty, where the instances are harder for it by more than the growth of its own budget makes up for, and falls away after that. Over the 22 configurations it stands at 10.75 per cent, against 3.52 for the 2-approximation and 2.53 for NEH, with a worst instance 67.41 per cent from the optimum, at ten jobs. Its cost is two to five orders of magnitude above the rules: 0.55 ms an instance at ten jobs and 1.65 s at sixty, against 8 ms for NEH and 0.02 ms for the 2-approximation, though still less than a fiftieth of the 80.1 s the model needs at that size. On this scale the search neither beats the rules nor approaches them cheaply; what it shows is the direction in which it moves, and that is read on the large scale. Tables 2 to 3 give the two ranges in full, and Figure 6 the course of the gap over the scale.

Table 2: The small scale, $a_j, b_j \in [1, 40]$: MILP, 2-approx., NEH, GA

n	MILP				2-approx.				NEH				GA			
	$WC_{\max}$		time (ms)		gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
10	2510.7	3192.0	5.1e1	8.0e1	4.9	14.2	1.0e−2	1.0e−2	**0.8**	8.7	8.0e−2	2.7e−1	16.8	67.4	5.5e−1	7.5e−1
15	3682.3	4746.0	1.4e2	2.8e2	2.7	8.7	1.0e−2	2.0e−2	**2.0**	8.0	1.8e−1	2.2e−1	19.0	42.0	2.0e0	3.0e0
20	4742.4	5759.0	3.8e2	8.9e2	4.6	9.5	1.0e−2	2.0e−2	**2.3**	9.7	3.9e−1	4.2e−1	14.2	34.2	1.0e1	1.1e1
25	5736.1	7260.0	7.3e2	2.3e3	3.8	10.8	1.0e−2	2.0e−2	**1.8**	9.2	6.9e−1	7.3e−1	11.5	27.9	2.6e1	2.9e1
30	6513.6	8892.0	1.3e3	2.4e3	4.4	9.3	1.0e−2	3.0e−2	**2.9**	9.3	1.0e0	1.0e0	14.7	43.6	6.0e1	6.4e1
35	7836.4	8931.0	3.0e3	1.1e4	**2.6**	9.7	2.0e−2	3.0e−2	2.7	7.7	2.0e0	2.0e0	15.0	30.7	1.2e2	1.3e2
40	8760.2	10153.0	5.1e3	1.6e4	3.1	7.7	2.0e−2	3.0e−2	**2.5**	8.7	3.0e0	3.0e0	8.4	21.2	2.4e2	2.6e2
45	9564.3	11076.0	1.3e4	3.8e4	3.6	8.0	2.0e−2	3.0e−2	**2.6**	7.1	4.0e0	4.0e0	6.0	17.0	4.0e2	4.4e2
50	10631.6	11774.0	2.0e4	6.2e4	2.7	7.2	2.0e−2	3.0e−2	**2.5**	7.1	5.0e0	5.0e0	6.9	21.5	6.6e2	7.3e2
55	11760.6	12992.0	3.7e4	1.8e5	3.0	6.4	2.0e−2	5.0e−2	**3.0**	8.1	6.0e0	6.0e0	3.3	10.8	1.0e3	1.1e3
60	12323.6	14703.0	8.0e4	3.0e5	**2.9**	8.9	2.0e−2	4.0e−2	3.3	6.7	8.0e0	8.0e0	3.9	10.1	1.7e3	1.7e3

Table 3: The small scale, $a_j, b_j \in [1, 80]$: MILP, 2-approx., NEH, GA

n	MILP				2-approx.				NEH				GA			
	$WC_{\max}$		time (ms)		gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
10	5433.0	7365.0	4.0e1	8.7e1	4.6	16.7	1.0e−2	1.0e−2	**0.6**	2.1	8.0e−2	1.6e−1	13.2	41.9	5.3e−1	5.8e−1
15	6941.7	9035.0	1.6e2	3.0e2	4.2	11.7	1.0e−2	2.0e−2	**2.2**	8.2	1.8e−1	2.0e−1	19.0	37.2	2.0e0	3.0e0
20	9029.8	10906.0	3.9e2	6.5e2	3.8	13.7	1.0e−2	2.0e−2	**1.3**	7.4	3.9e−1	4.9e−1	12.6	37.3	1.0e1	1.1e1
25	10971.5	13013.0	6.9e2	1.4e3	4.0	11.1	1.0e−2	3.0e−2	**3.1**	10.7	7.1e−1	7.5e−1	16.0	27.1	2.6e1	2.8e1
30	12832.5	15232.0	1.9e3	9.4e3	3.9	8.8	1.0e−2	2.0e−2	**3.4**	7.9	1.0e0	1.0e0	12.9	30.6	5.8e1	6.6e1
35	15171.0	16940.0	3.3e3	1.1e4	3.8	7.8	2.0e−2	2.0e−2	**3.1**	11.3	2.0e0	2.0e0	10.0	19.8	1.2e2	1.2e2
40	16912.4	19513.0	8.3e3	2.5e4	**2.9**	11.3	2.0e−2	3.0e−2	3.1	11.0	3.0e0	3.0e0	10.4	29.7	2.4e2	2.6e2
45	19067.3	23062.0	9.2e3	1.8e4	3.0	7.3	2.0e−2	4.0e−2	**2.6**	7.8	4.0e0	4.0e0	7.1	23.5	4.1e2	4.4e2
50	20795.4	23505.0	2.2e4	5.1e4	3.5	9.6	2.0e−2	3.0e−2	**3.4**	9.7	5.0e0	5.0e0	5.5	15.0	6.6e2	7.1e2
55	22517.8	27573.0	5.5e4	1.4e5	**2.8**	9.3	2.0e−2	5.0e−2	3.7	10.1	6.0e0	6.0e0	6.8	17.6	1.0e3	1.2e3
60	25266.1	29614.0	4.4e4	1.0e5	**2.7**	8.4	3.0e−2	4.0e−2	2.8	6.9	8.0e0	8.0e0	3.2	8.9	1.7e3	1.8e3

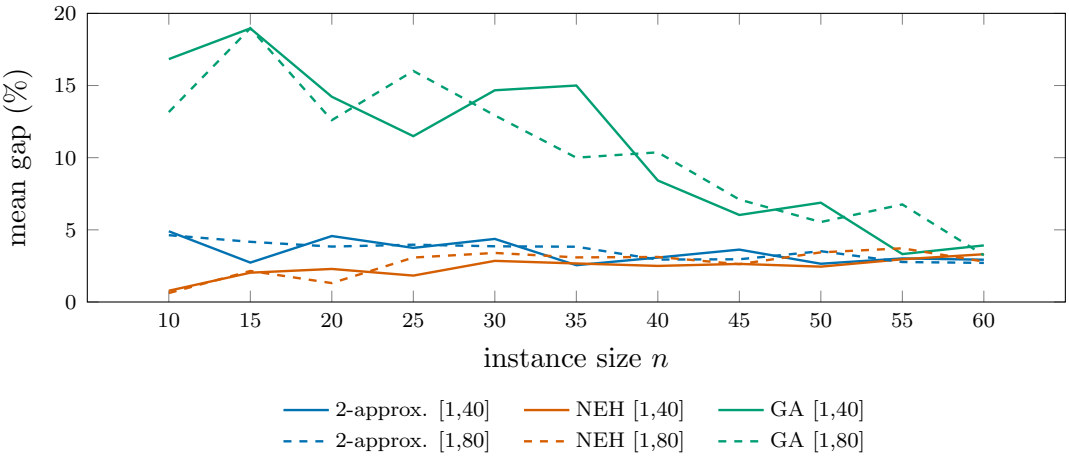


Figure 6: Mean relative gap of the 2-approximation and the two heuristics on the small scale, against the optimum the model returned. A solid line is $a_j, b_j \in [1, 40]$, a dashed line $a_j, b_j \in [1, 80]$.

On this scale the three methods end within a point of one another, the search having travelled the whole way to join the two rules. What still separates them is the price: the model is exact but close to the limit of its budget, NEH is the closer of the two rules at the bottom of the scale and the 2-approximation at the top, and the search pays two to five orders of magnitude above the rules for a gap of the same order. What that price buys is read on the large scale.

6.3 The large scale

On the large scale the three methods separate for good. The rules are flat: the 2-approximation is the closest on average at 1.81 per cent over the 32 configurations, its mean never above 3.20 per cent and its worst instance 7.57 per cent away, and NEH is the furthest at 4.40 per cent, its gap hardly moving over the scale – 5.02 at a hundred jobs against 3.60 at four hundred on $[1, 40]$, and 5.39 against 4.47 on $[1, 80]$, with a worst instance 12.64 per cent away. The genetic algorithm is the only method that overtakes the other two, and its gap falls steeply as the instance grows: 10.88 per cent at a hundred and eighty jobs, 5.49 at two hundred, 2.09 at two hundred and forty and 0.22 at four hundred on $[1, 40]$, and 6.99, 6.34, 2.09 and 0.13 per cent on $[1, 80]$. It passes NEH at 240 jobs and the 2-approximation at about 260 on $[1, 80]$ and 280 on $[1, 40]$, and over the 640 instances of the scale it attains the best value on 318 of them, against 283 for the 2-approximation and 42 for NEH, a tie counted for every method that attains the value. What it costs is time and the bottom of the scale: 70.8 s an instance on average, against 0.076 ms for the 2-approximation and 0.76 s for NEH, and 350.4 s on its slowest instance of the 600 s a run allows; its worst configuration is early in the scale, 10.88 per cent at a hundred and eighty jobs, and its worst instance at a hundred, 28.17 per cent.

The two ranges agree on the ordering of the two rules and disagree on the search. The 2-approximation is closer on the wider range, 1.70 per cent against 1.92, and NEH further from it, 4.79 against 4.02, so that the 2-approximation leads NEH on both ranges. The search overtakes NEH only on $[1, 80]$, at 3.65 per cent against 4.79, and does not on $[1, 40]$, where it stands at 4.28 against 4.02. Tables 4 to 5 give the two ranges in full, and Figure 7 the course of the gap over the scale.

Table 4: The large scale, $a_j, b_j \in [1, 40]$: 2-approx., NEH, GA

n	2-approx.				NEH				GA			
	gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
100	**3.2**	7.6	3.0e−2	7.0e−2	5.0	7.7	3.5e1	3.6e1	10.6	28.2	1.4e3	1.4e3
120	**2.9**	5.1	3.0e−2	5.0e−2	4.3	11.0	6.1e1	6.7e1	9.7	21.4	2.2e3	2.3e3
140	**2.7**	6.1	3.0e−2	4.0e−2	4.1	6.2	9.2e1	9.7e1	10.8	23.2	3.2e3	3.3e3
160	**1.9**	3.7	4.0e−2	4.0e−2	4.6	7.1	1.3e2	1.4e2	9.6	20.5	4.6e3	4.7e3
180	**2.4**	5.3	4.0e−2	7.0e−2	4.1	7.3	1.9e2	2.0e2	10.9	20.4	6.4e3	6.6e3
200	**2.1**	5.1	5.0e−2	6.0e−2	3.9	8.5	2.6e2	2.7e2	5.5	14.2	1.1e4	1.1e4
220	**2.0**	5.1	6.0e−2	1.0e−1	3.9	5.9	3.5e2	3.5e2	4.3	10.8	1.7e4	1.8e4
240	**1.7**	3.5	6.0e−2	1.0e−1	4.2	7.2	4.5e2	4.6e2	2.1	8.1	2.6e4	2.7e4
260	**1.7**	3.9	7.0e−2	8.0e−2	4.0	6.7	5.7e2	5.8e2	1.7	6.0	3.9e4	4.0e4
280	1.6	4.6	7.0e−2	8.0e−2	4.0	7.3	7.2e2	7.4e2	**0.8**	3.8	5.8e4	5.9e4
300	2.1	5.4	8.0e−2	1.1e−1	4.1	6.2	8.9e2	9.1e2	**0.7**	2.0	7.7e4	7.9e4
320	1.4	2.7	9.0e−2	1.4e−1	3.6	6.1	1.1e3	1.1e3	**0.6**	2.4	1.0e5	1.0e5
340	1.3	4.0	1.0e−1	1.5e−1	3.5	5.5	1.3e3	1.3e3	**0.4**	1.9	1.3e5	1.3e5
360	1.2	2.9	1.1e−1	1.4e−1	3.7	7.2	1.6e3	1.6e3	**0.2**	0.9	1.6e5	1.6e5
380	1.3	3.1	1.2e−1	1.5e−1	3.8	6.0	1.9e3	2.1e3	**0.4**	1.6	2.1e5	2.2e5
400	1.5	3.0	2.0e−1	8.9e−1	3.6	4.9	2.4e3	2.9e3	**0.2**	1.1	2.8e5	3.3e5

Table 5: The large scale, $a_j, b_j \in [1, 80]$: 2-approx., NEH, GA

n	2-approx.				NEH				GA			
	gap		time (ms)		gap		time (ms)		gap		time (ms)	
	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
100	**2.8**	5.3	3.0e−2	5.0e−2	5.4	12.6	3.6e1	3.7e1	9.0	23.6	1.4e3	1.5e3
120	**2.4**	4.7	3.0e−2	6.0e−2	4.6	6.5	6.0e1	6.6e1	9.1	20.9	2.2e3	2.2e3
140	**2.2**	6.0	3.0e−2	4.0e−2	4.8	8.4	9.1e1	9.3e1	7.2	15.1	3.1e3	3.2e3
160	**2.6**	5.5	4.0e−2	4.0e−2	4.0	7.5	1.4e2	1.4e2	10.2	21.9	4.6e3	4.7e3
180	**2.1**	4.7	4.0e−2	5.0e−2	4.7	7.2	1.9e2	2.0e2	7.0	17.1	6.4e3	6.7e3
200	**1.9**	6.6	5.0e−2	7.0e−2	5.0	8.6	2.6e2	2.7e2	6.3	17.5	1.1e4	1.1e4
220	**1.4**	3.0	5.0e−2	6.0e−2	5.1	7.6	3.4e2	3.5e2	2.5	7.3	1.7e4	1.8e4
240	**1.3**	2.4	6.0e−2	7.0e−2	4.6	8.0	4.5e2	4.6e2	2.1	6.8	2.6e4	2.7e4
260	1.9	4.9	7.0e−2	8.0e−2	5.6	8.0	5.7e2	5.8e2	**1.9**	5.7	3.9e4	4.0e4
280	1.4	3.1	7.0e−2	9.0e−2	3.9	6.5	7.2e2	7.3e2	**0.7**	2.4	5.8e4	5.9e4
300	1.2	2.9	9.0e−2	1.3e−1	4.9	6.7	8.9e2	9.1e2	**1.1**	3.8	7.7e4	7.8e4
320	1.2	3.1	9.0e−2	1.2e−1	4.3	6.0	1.1e3	1.1e3	**0.5**	1.8	9.9e4	1.0e5
340	1.1	3.0	1.0e−1	1.2e−1	4.6	6.6	1.3e3	1.3e3	**0.2**	1.0	1.3e5	1.3e5
360	1.2	2.2	1.1e−1	1.3e−1	5.4	6.8	1.6e3	1.6e3	**0.2**	1.4	1.6e5	1.6e5
380	1.6	3.3	1.2e−1	1.3e−1	5.2	7.3	1.9e3	2.0e3	**0.4**	1.8	2.1e5	2.1e5
400	1.1	3.7	2.0e−1	4.3e−1	4.5	7.7	2.7e3	3.2e3	**0.1**	0.7	3.1e5	3.5e5

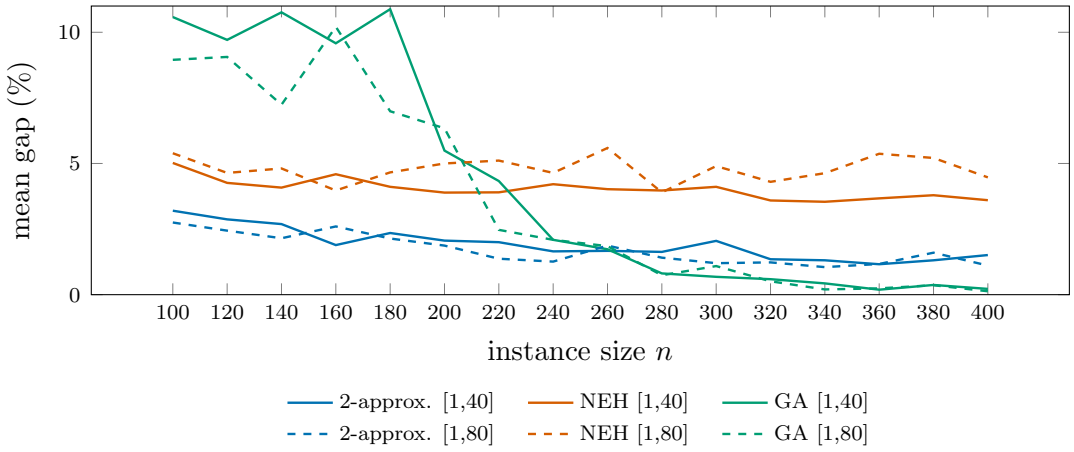


Figure 7: Mean relative gap of the 2-approximation and the two heuristics on the large scale, against the lower bound LB. A solid line is $a_j, b_j \in [1, 40]$, a dashed line $a_j, b_j \in [1, 80]$.

At this scale, then, the rules stay within a few points of the bound while the search falls through both, and it is there, and only there, that the population buys a value the rules do not reach. What the study reads is a division of labour rather than a ranking. The 2-approximation is the cheapest method by six orders of magnitude and averages 3.52

per cent above the optimum on the small scale, with a worst instance at 16.71, and 1.81 per cent above LB on the large one, with a worst at 7.57, which says that the factor of two of Theorem 4.2 is loose by more than an order of magnitude on instances of this kind. What it cannot do is read anything but the weights, and that is the ground NEH takes: NEH's initial order reads the processing times alone, and each insertion is judged by the weighted objective, which makes it the closest method at the bottom of the small scale and the furthest at the top of the large one, where its gap does not close as the instance grows.

The two methods that carry the study beyond a rule sit at opposite ends of it. The dynamic program is the only one that certifies an optimum, and it does so cheaply where it applies, at 1.47 s for forty jobs on $[1, 40]$, but its applicability is the restriction: two weight classes, at most a hundred jobs, and a third class that exhausts the memory of the machine before the search starts. The genetic algorithm is the reverse. It certifies nothing, it costs two to five orders of magnitude more than the rules on the small scale and more than ninety times NEH on the large one, and it is nevertheless the one that beats both rules from about 240 to 280 jobs on, best on 318 of the 640 large instances against 283 and 42. Neither is read on the instances of the other, the dynamic program on a set of its own at $K = 2$ and the search on instances that carry up to thirty weight classes, so what separates them is not a common benchmark but the question each answers: whether the optimum can be certified at all, or whether a schedule that beats every rule can be built at the sizes that arise. The reason the search needs the room is the start it is given: Johnson's rule carries no weight, so its early generations are spent building the objective into the schedule before it can profit from it.

Two properties of the comparison are worth keeping in view. The large-scale gaps are distances to LB, which bounds the optimum from below through the three prefix bounds of (31) and is tight only where the heaviest jobs are also the earliest ones; what they order reliably is therefore the methods and not their absolute quality. And the answers depend on the processing-time range: the average advantage of the search over NEH, 3.97 against 4.40 per cent, is earned on $[1, 80]$ alone.

One lever is left unpulled on the side of the search. Seeding its population with the order the 2-approximation returns costs microseconds and carries the weights the Johnson order lacks, which is the cheap way to move the size at which the search overtakes the rules.

7 Conclusion

This paper has shown that the two-machine flow shop with the maximum weighted completion time as its objective is strongly NP-hard, and that it admits a structure on which exact computation becomes possible: an optimal schedule is a permutation, it splits into one block per distinct weight, Johnson's order holds inside a block, and the blocks follow the weights in non-increasing order. The dynamic program built on that structure prunes dominated states and runs in $O(nK2^K V^{3K-3})$ time, the 2-approximation reads the weights alone and runs in $O(n \log n)$ time, the approximation scheme attains a $(1+\varepsilon)$ guarantee for a constant number of weights, and the two heuristics, an adapted NEH and a genetic algorithm, cover the sizes at which none of the exact methods reaches.

The computational study places each of them in its range. The mixed integer model settles all 440 small instances to optimality, and at $K = 2$ the dynamic program certifies the optimum up to a hundred jobs on $[1, 40]$ and to eighty on $[1, 80]$, where it is the 600 s limit and not its correctness that stops it. Against the optimum the 2-approximation stays within 3.5 per cent on average and NEH within 2.5; against the lower bound on the large scale the same two rules stay within 1.8 and 4.4 per cent, while the genetic algorithm, the weakest of the three on small instances, is the only method that overtakes both rules, passing them at about 240 to 280 jobs. The practical reading is a division of labour rather than a ranking: the rules are cheap and dependable, the exact methods certify within their reach, and the search buys values that neither rule reaches.

Three extensions follow from the limits of the study, one for each of its methods. The first concerns the number of machines: the mixed integer formulation of Section 2 is written for two machines and generalises, the assignment constraints and the completion-time recursions carrying over to $m > 2$ machines with one sequence per machine in place of the single permutation, since the permutation property that holds for two machines fails in general with three or more. What the extension costs is size, a set of sequencing variables for every machine, and not principle.

The second concerns the number of weight classes. The dynamic program is stopped at $K = 3$ by its state space, bounded by $2^K V^{3K-3}$, which exhausts the memory of the machine before the search starts. Branch and bound is the natural successor there, and two of its ingredients are already in the paper: the block structure gives the branching rule, over the order of the blocks and over the jobs within one, and the three thresholds LB_1, LB_2 and LB_3 of Section 4.3 give a bound at every node. What such a procedure would keep is part of the certification the program gives at $K = 2$, on instances the program cannot reach.

The third concerns the setting. Both heuristics read only the processing times and the weights, so they carry over to the distributed flow shop, where several identical flow shops process the jobs and the assignment of a job to a shop is added to the sequencing decision: the insertion test of NEH can be taken shop by shop, and the chromosome of the genetic

algorithm can carry the assignment beside the permutation. Variants that keep a two-machine shop inside a larger problem admit the same treatment. Each of the three moves extends the reach of a method without changing the structure it exploits.

References

Becker, T., Neufeld, J. S., and Buscher, U. (2025). The distributed flow shop scheduling problem with inter-factory transportation. *European Journal of Operational Research*, 322(1):39–55.

Deb, K., Pratap, A., Agarwal, S., and Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197.

Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of $\mathcal{NP}$-Completeness*. W. H. Freeman and Company, San Francisco, CA.

Garey, M. R., Johnson, D. S., and Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1(2):117–129.

Hall, L. A. (1994). A polynomial approximation scheme for a constrained flow-shop scheduling problem. *Mathematics of Operations Research*, 19(1):68–85.

Han, B., Pan, Q.-K., and Gao, L. (2024). A cooperative iterated greedy algorithm for the serial distributed permutation flowshop scheduling problem. *International Journal of Production Research*, 62(12):4245–4272.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.

Koutecký, M. and Zink, J. (2025). Complexity of scheduling few types of jobs on related and unrelated machines. *Journal of Scheduling*, 28(1):139–156.

Liu, L. and Urgo, M. (2024a). A branch-and-bound approach to minimise the value-at-risk of the makespan in a stochastic two-machine flow shop. *International Journal of Production Research*, 62(6):2107–2123.

Liu, L. and Urgo, M. (2024b). Robust scheduling in a two-machine re-entrant flow shop to minimise the value-at-risk of the makespan: branch-and-bound and heuristic algorithms based on Markovian activity networks and phase-type distributions. *Annals of Operations Research*, 338(1):741–764.

Nawaz, M., Enscore, E. E., and Ham, I. (1983). A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. *Omega*, 11(1):91–95.

Pinedo, M. L. (2022). *Scheduling: Theory, Algorithms, and Systems*. Springer, Cham, Switzerland, 6 edition.

Puka, R., Skalna, I., Duda, J., and Stawowy, A. (2024). Deterministic constructive vN-NEH+ algorithm to solve permutation flow shop scheduling problem with makespan criterion. *Computers & Operations Research*, 162:106473.

Qiao, Y., Wu, N., He, Y., Li, Z., and Chen, T. (2022). Adaptive genetic algorithm for two-stage hybrid flow-shop scheduling with sequence-independent setup time and no-interruption requirement. *Expert Systems with Applications*, 208:118068.

von Aspern, M., Buld, F., Klein, N., and Pinedo, M. (2025). Flow shops with reentry: The total weighted completion time objective. *Journal of Scheduling*, 28(6):585–600.

Wang, J., Wang, G. G., Tian, F. M., Gong, D. W., and Pedrycz, W. (2023). Solving energy-efficient fuzzy hybrid flow-shop scheduling problem at a variable machine speed using an extended NSGA-II. *Engineering Applications of Artificial Intelligence*, 121:105977.

Ying, K.-C., Lin, S.-W., and Chen, K.-F. (2025). Minimising makespan in distributed assembly hybrid flowshop scheduling problems. *International Journal of Production Research*, 63(5):1674–1691.